Escuela Politécnica Superior

Project Rogue, desarrollo de un juego para la Game Boy



Grado en Ingeniería Multimedia

Trabajo Fin de Grado

Autor:
Enrique Vidal Cayuela

Tutor/es:
Francisco José Gallego Durán

Julio 2020

Universitat d'Alacant
Universidad de Alicante

Project Rogue

Videojuego realizado para la Game Boy

Autor
Enrique Vidal Cayuela (alumno)

Directores
Francisco José Gallego Durán (tutor)
Dpto: B161 Ciencia de la Computación e Inteligencia Artificial



Grado en Ingeniería Multimedia



Escuela
Politécnica
Superior



ALICANTE, 30 de julio de 2020

Preámbulo

Durante los años cursados en la carrera de Ingeniería Multimedia he adquirido multitud de conocimientos: programar en diferentes lenguajes, modelar en 3D, gestionar proyectos, etc. Pero lo que considero más importante es que te enseñan a entender las bases de todo aquello sobre lo que se trabaje como requisito para ser un buen ingeniero.

Por eso mismo, hoy en día donde el desarrollo de los videojuegos se ha vuelto mucho más sencillo gracias, en gran medida, a herramientas como Unity o Unreal Engine, se ha decidido realizar un videojuego desde la base para una de las consolas retro más vendidas hasta la fecha, la Game Boy de Nintendo.

Teniendo en cuenta que por muy complejos que sean los juegos actuales, todos parten de una misma base. La finalidad de este proyecto es la de aprender las bases de la programación de un videojuego tal y como se hacía hace más de 25 años.

Este proyecto simboliza el fin de un periodo de formación como ingeniero, donde he conocido a muchas personas con la que he compartido buenos y malos momentos. He aprendido mucho, tanto de las materias impartidas como de mi persona y del trabajo en equipo. Y gracias a todo esto he podido realizar uno de los sueños de mi infancia, desarrollar un videojuego.

Este proyecto también inicia un nuevo camino donde poder aplicar todo lo aprendido y seguir aprendiendo. Por eso mismo, quiero agradecer a todas aquellas personas que me han apoyado y animado durante el desarrollo, y que espero lo sigan haciendo en el futuro.

Gracias a mi tutor Francisco José Gallego por guiarme y aconsejarme durante la realización de este proyecto.

Gracias a todos mis amigos y compañeros, por apoyarme y soportarme en los periodos más duros, sobre todo durante las semanas que he pasado en casa sin poder salir.

Por último y los más importantes, gracias a mis padres José Alfredo y María Dolores, que me han motivado y aportado todo lo necesario desde pequeño para que pueda cumplir este objetivo en mi vida.

Si consigo ver más lejos
es porque he conseguido auparme
a hombros de gigantes

Isaac Newton.

Índice general

1. Introducción **1**

2. Marco Teórico **3**
2.1. Qué es un videojuego . . . 3
2.2. La Game Boy . . . 3
2.2.1. Historia . . . 3
2.2.2. Versiones de la Game Boy . . . 5
2.2.3. Características Técnicas . . . 8
2.2.4. Procesador Sharp LR35902 . . . 10
2.2.5. Registros . . . 10
2.2.6. Instrucciones . . . 11
2.2.7. Mapa de Memoria . . . 12
2.2.8. Memoria ROM . . . 14
2.2.9. Memoria de Vídeo . . . 15
2.3. Juegos que han influenciado al proyecto . . . 17
2.3.1. Rogue . . . 18
2.3.2. Shiren the Wanderer GB . . . 19
2.3.3. The Legend of Zelda: Link's Awakening . . . 20

3. Objetivos **23**

4. Metodología **25**
4.1. Gestión del proyecto . . . 25
4.1.1. Trello . . . 25
4.1.2. GitHub . . . 25

5. Game Design Document **29**
5.1. Resumen . . . 29
5.1.1. Introducción . . . 29
5.1.2. Características clave . . . 29
5.2. Mecánicas Básicas . . . 30
5.2.1. Jugador: . . . 30
5.2.2. Enemigos . . . 36
5.2.3. Objetos . . . 38
5.2.4. Mazmorra . . . 38
5.2.5. Base . . . 39
5.2.6. GUI . . . 40
5.3. Controles . . . 42

5.4. Estilo visual . . . 43
5.5. Diégesis . . . 43
5.5.1. Historia: . . . 43
5.5.2. Protagonista/s: . . . 44
5.5.3. PNJs hostiles: . . . 45
5.5.4. PNJs amistosos: . . . 46
5.6. Herramientas de Desarrollo . . . 47
5.6.1. Ensamblador . . . 48
5.6.2. Editor de texto . . . 49
5.6.3. Editor de imágenes . . . 50
5.6.4. Editor de tiles y sprites . . . 50
5.6.5. Editor de mapas . . . 51
5.6.6. Herramientas de testeo y depuración . . . 51

6. Desarrollo **57**
6.1. Iteración 1: Elección de Ensamblador . . . 57
6.2. Iteración 2: Makefile y Código Inicial . . . 59
6.2.1. Makefile . . . 59
6.2.2. Código Inicial . . . 60
6.3. Iteración 3: Dibujado por Pantalla . . . 61
6.4. Iteración 4: Mapas, Entidades y Físicas Iniciales . . . 65
6.4.1. Mapas . . . 65
6.4.2. Entidades . . . 66
6.4.3. Físicas Iniciales . . . 67
6.5. Iteración 5: Mapa con carga dinámica . . . 68
6.6. Iteración 6: Modelo ECS . . . 69
6.7. Iteración 7: Tiles Avanzados y Físicas Finales . . . 71
6.7.1. Tiles Avanzados . . . 72
6.7.2. Físicas Finales . . . 72
6.8. Iteración 8: Enemigos, IA y Combate . . . 73
6.8.1. Enemigos . . . 73
6.8.2. Inteligencia Artificial . . . 74
6.8.3. Combate . . . 76
6.9. Iteración 9: HUD y Menús . . . 77
6.9.1. HUD . . . 77
6.9.2. Menús . . . 79
6.10. Iteración 10: Objetos y Equipo . . . 80
6.10.1. Objetos . . . 80
6.10.2. Equipo . . . 81
6.11. Iteración 11: Generación Procedimental . . . 82

7. Conclusiones **85**

Bibliografía **87**

A. Anexo **89**
A.1. Argumentos de RGBASM 89
A.2. Argumentos de RGBLINK 89
A.3. Argumentos de RGBFIX 89

Índice de figuras

2.1. Imagen Game&Watch Ball 4
2.2. Consola Atari Lynx. [1] 4
2.3. Consola Sega Game Gear.[2] 5
2.4. Consola Game Boy Original.[3] 6
2.5. Consola Game Boy Pocket.[4] 7
2.6. Consola Game Boy Light.[5] 7
2.7. Consola Game Boy Colorl.[6] 8
2.8. Esquema de la relación entre el fondo y la vista. 16
2.9. Esquema que muestra el funcionamiento de las diferentes partes de la memoria de vídeo. 18
2.10. captura del vieojuego Rogue.[7] 19
2.11. captura del vieojuego Shiren the Wanderer GB. 20
2.12. captura del vieojuego The Legend of Zelda: Link's Awakening. 21

4.1. captura del tablero de Trello. 26
4.2. captura de la página de GitHub. 27

5.1. Boceto que representa el movimiento del jugador y los enemigos. 30
5.2. Boceto que representa la elección de dirección del jugador. 31
5.3. Boceto que representa las colisiones del jugador. 31
5.4. Boceto que representa la acción de atacar a un enemigo. 32
5.5. Boceto que representa la acción de pasar turno atacando al aire. 32
5.6. Boceto que representa los diferentes tipos de ataques mágicos. 33
5.7. Boceto que representa el consumo de puntos de magia. 33
5.8. Boceto que representa el estado alterado veneno. 34
5.9. Boceto que representa el estado alterado parálisis. 34
5.10. Boceto que representa la muerte del jugador. 35
5.11. Boceto que representa los objetos dejados por el jugador. 35
5.12. Boceto que representa el diseño de la mazmorra. 39
5.13. Boceto que representa el diseño de la base. 40
5.14. Boceto que representa el diseño del HUD. 41
5.15. Boceto que representa el diseño del menú de pausa. 41
5.16. Boceto que representa el diseño del inventario del jugador. 42
5.17. Boceto que representa el diseño del personaje principal. 44
5.18. Boceto que representa el diseño del Murciélago Gigante. 45
5.19. Boceto que representa el diseño del Ojo Diabólico. 45
5.20. Boceto que representa el diseño del Mago Oscuro. 46

5.21. Boceto que representa el diseño del Elemental de Roca. 46
5.22. Boceto que representa el diseño de la Mujer Misteriosa. 47
5.23. Boceto que representa el diseño del Mercader. 47
5.24. Extracto del código ensamblador del proyecto. 48
5.25. Captura de pantalla del Visual Studio Code. 49
5.26. Captura de pantalla de Krita. 50
5.27. Captura de pantalla del Game Boy Tile Designer. 51
5.28. Captura de pantalla del Game Boy Map Builder. 52
5.29. Captura de pantalla del debugger de BGB. 53
5.30. Captura de pantalla del apartado BG map del emulador BGB. 54
5.31. Captura de pantalla del apartado Tiles del emulador BGB. 54
5.32. Captura de pantalla del apartado OAM del emulador BGB. 55
5.33. Captura de pantalla del apartado Pelettes del emulador BGB. 55
5.34. Fotografía de la flashcart EZ-Flash jr. 56

6.1. Diagrama de la linea de ensamblado 58
6.2. Diagrama del funcionamiento de un makefile 59
6.3. Esquema de la codificación de un tile 62
6.4. Esquema de cargar un tile en VRAM 63
6.5. Conversión de 8x8px a 16x16px 64
6.6. Esquema del dibujado una porción del tilemap en la vista 68
6.7. Esquema de la carga dinámica del mapa en el eje Y 70
6.8. Esquema de los índices de division de los tiles 73
6.9. Diagrama del Árbol de Decisiones 75
6.10. Esquema de las salas y las salidas del mapa 76
6.11. Esquema que muestra la generación procedimental del proyecto. 83

7.1. Fotografía del juego siendo ejecutado en una Game Boy Advance SP. . . . 86

Índice de tablas

2.1. Especificaciones técnicas de la consola Game Boy y sus revisiones.[8] 9
2.2. Distribución del mapa de memoria de la Game Boy. 13
2.3. Mapeado de la memoria ROM 14
2.4. Distribución de la memoria de vídeo. 15

6.1. Funciones del control LCD [0xFF40] 61

A.1. Argumentos de RGBASM. 90
A.2. Argumentos de RGBLINK. 91
A.3. Argumentos de RGBFIX. 92

Índice de Listados

6.1. RGBASM Código de Ejemplo 58
6.2. RGBLINK Código de Ejemplo 58
6.3. RGBFIX Código de Ejemplo 58

A.1. Estructura del comando rgbasm 89
A.2. Estructura del comando rgblink 89
A.3. Estructura del comando rgblink 89

1. Introducción

El videojuego ha pasado por multitud de etapas hasta convertirse en lo que es hoy en día. En sus inicios, el videojuego no se veía como algo más que mero entretenimiento para los niños. Sin embargo, conforme ha ido avanzando la tecnología, el desarrollo de los videojuegos ha ido facilitándose y expandiéndose a un público cada vez más mayoritario y diverso.

Desde videoconsolas y ordenadores hasta móviles y tablets, los videojuegos han sabido adaptarse al medio que les rodea, lo que ha permitido que los desarrolladores creen experiencias más diversas y personales, capaces de entrar en todos los ámbitos de la vida. El sector de los videojuegos ya no es solo un mero entretenimiento para niños, muchos críticos lo posicionan como una nueva forma de arte que combina otras como la pintura, el cine o la música y añade una nueva dimensión, la interacción.

En esta interacción con el medio es donde los desarrolladores han innovado hasta el punto en que ahora los videojuegos se pueden usar como herramienta en otros sectores como la enseñanza, potenciando la creatividad y la cooperación; para tratamientos médicos como el autismo, o incluso para industria del cine, donde se están empezando a usar motores gráficos de videojuegos como Unreal Engine para simular entornos y abaratar costes. Los videojuegos también se han adentrado en el mundo de los deportes, creando una nueva forma de los mismos, los e-sports. En estos, equipos de todo el mundo se enfrentan a partidas de diversos juegos, creando un movimiento de fans capaz de competir con muchos deportes actuales.

Gran parte de la fama que tienen hoy día los videojuegos se debe al desarrollo independiente. Este ha estado siempre vigente, pero no ha sido hasta que han surgido herramientas como Unreal Engine o Unity entre otras, que ha explotado el movimiento independiente de desarrollo de videojuegos. Estas herramientas han facilitado la creación de los mismos, hasta el punto en que con mínimos conocimientos de programación se puede empezar a crear un primer videojuego e incluso ponerlo a la venta en múltiples plataformas.

Facilitar el desarrollo de esta forma permite que muchas personas que antes no podían, empiecen a desarrollar sus ideas. Pero por otro lado, se pierde en conocimiento de cómo funcionan internamente o qué es lo que están modificando, lo que provoca una dependencia hacia la herramienta y sus limitaciones o sin ir más lejos, la incapacidad de corregir algún error provocado por la propia plataforma. Debido a esta falta de conocimiento sobre la base de la programación de videojuegos, el objetivo de este proyecto es crear un videojuego para la consola Game Boy de Nintendo.

Los videojuegos actuales se ejecutan en consolas o ordenadores con arquitecturas muy complejas y se necesitan muchos conocimientos para poder sacarles todo el partido. Por eso la elección de una consola retro como la Game Boy es un punto de partida ideal,

pues son lo suficientemente sencillas para aprender su funcionamiento y sacarle todo el partido.

El objetivo de esta memoria es describir cómo ha sido el desarrollo del videojuego así como el análisis previo y la metodología empleada para el mismo. Pero antes de eso se debe empezar con un poco de contexto sobre lo que se va a desarrollar en el proyecto.

2. Marco Teórico

En este apartado se van a desarrollar todos aquellos conceptos esenciales relacionados con el proyecto así como todos los referentes previos al mismo, empezando por la definición de videojuego.

2.1. Qué es un videojuego

Para el usuario, un videojuego se podría definir como un juego electrónico en el que una o más personas interactúan, por medio de un controlador, con un dispositivo que muestra imágenes de vídeo. Pero para un desarrollador, esta definición se queda algo corta.

Para un desarrollador, sería necesario ampliar un poco más la definición. Un videojuego es un programa de software que es procesado por un hardware con entradas y salidas llamado plataforma. Dicho programa contiene todos los datos referentes al videojuego: información, lógica, gráficos y sonido; y se almacena en un contenedor ya sea un cartucho, un CD o incluso un disco duro aunque sea de manera temporal.

Toda la información referente a la definición del videojuego se puede encontrar en la siguiente referencia [1].

Esta información es muy relevante, pues un videojuego como programa software que es, está ligado completamente al sistema sobre el que se ejecute. Esta relación permite que el videojuego pueda aprovechar todas las cualidades de la plataforma, sin embargo también tendrá que contemplar las limitaciones de la misma.

Dicho esto, es el momento de analizar las características que tiene la consola Game Boy de Nintendo.

2.2. La Game Boy

2.2.1. Historia

La Game Boy es una videoconsola portátil de 8 bits desarrollada y manufacturada por Nintendo lanzada en Japón y Norte America en el año 1989, mientras que a Europa no llegaría hasta Septiembre del 1990. Uno de los principales responsables del desarrollo de esta consola es Gunpei Yokoi, quien también fue el creador de las Game & Watch [2.1], una serie de 59 consolas capaces de jugar un juego distinto cada una de ellas.

[1] Autor: ThePViana.
Fuente: `https://en.wikipedia.org/wiki/File:Game_%26_Watch.png`



Figura 2.1.: Primer modelo de Game & Watch con el juego Ball.[1]

Lo que diferencia a la Game Boy de sus predecesoras las Game & Watch es su capacidad de cambiar de juego mediante el uso de cartuchos, donde cada uno de estos podía contener uno o varios juegos. Otra de las diferencias es la pantalla, donde las Game & Watch empleaban un LCD con posiciones fijas para los gráficos, la Game Boy apostó por una pantalla LCD con tecnología DMC (dot-matrix screen) que le permitía dibujar puntos o píxeles por pantalla en un total de cuatro colores, siendo estos una escala de verdes de más claro a más oscuro.



Figura 2.2.: Consola Atari Lynx. [2]

[2]Autor Evan Amos.
Fuente: `https://commons.wikimedia.org/wiki/File:Atari-Lynx-Handheld-Angled.jpg`
[3]Autor: Fabien1309.
Fuente: `https://commons.wikimedia.org/wiki/File:Sega_gamegear.jpg`



Figura 2.3.: Consola Sega Game Gear.[3]

Estas características técnicas pusieron a la Game Boy en una clara desventaja inicial, pues en un momento donde los videojuegos ya estaban en los 16 bits, con miles de colores distintos y a pocos años del salto al 3D, la portatil de Nintendo podía verse anticuada. A eso no le ayudó que en el mismo año de lanzamiento apareciera la Atari Lynx [2.2], que al contrario que la Game Boy, tenía pantalla a color y retroiluminada. También aparecería otro rival un año después por parte de Sega, la Game Gear [2.3], que también disponía de pantalla a color y con iluminación.

Pese a estar en una desventaja técnica, la Game Boy consiguió hacerse con la gran mayoría del mercado, siendo hasta la fecha, la consola portátil más vendida teniendo en cuenta sus revisiones de las que se hablará más adelante. Esto fue gracias a la gran duración de la batería, pues con solo 4 pilas AA podía durar hasta 30 horas de juego, mientras que la competencia, utilizando 6 pilas AA solo duraban unas 5 horas, lo que hacía que la Game Boy fuera la elección preferida, sobre todo para viajes largos o largas sesiones de juego. Otro de los motivos más importantes fue el catálogo de videojuegos, que con joyas como Tetris, Super Mario Land o Pokémon, hicieron que muchos jugadores se decantaran por la misma.

2.2.2. Versiones de la Game Boy

Otra de las grandes apuestas que hizo Nintendo con la Game Boy fue la de sacar diferentes revisiones que añaden funcionalidades o mejoran aspectos de la original. Por eso mismo, en este apartado se van a analizar las características de cada una de las revisiones que ha tenido la Game Boy partiendo desde la original.

Game Boy Original

Como ya se ha mencionado anteriormente, la Game Boy original [2.4] era una consola de 8 bits que salió al mercado en el año 1989. Esta primera versión cuenta con una pantalla capaz de mostrar una escala de cuatro tonos de verde sin iluminación, consta de

8 botones, 4 de ellos unidos en una cruceta de dirección, y los cuatro restantes divididos en botones de acción (A y B) y botones de select y start. También cuenta con un altavoz mono, con una rueda en el lateral para ajustar el volumen, un jack de audio para auriculares, un ajuste de contraste para la pantalla, un interruptor de encendido, un conector para los cartuchos de juego o Game Packs, un puerto Link para la conexión entre consolas y un espacio para poner las 4 pilas AA que usa para funcionar. La batería de este modelo puede durar hasta 30 horas de juego.



Figura 2.4.: Consola Game Boy Original.[4]

Game Boy Pocket

La Game Boy Pocket fue la primera revisión que se hizo de la Game Boy original. Esta fue lanzada al mercado en 1996 para Japón y Estados Unidos, y en 1997 para el resto del mundo, siendo la principal diferencia respecto a la original el tamaño, pues es considerablemente más pequeña. El resto de diferencias son: la paleta de colores pasa de ser un gradiente de 4 tonos de verde a una escala de grises, la batería pasa de durar unas 30 horas a tan solo unas 10 debido a las baterías que usa, que pasan de ser 4 pilas AA a ser tan solo 2 pilas AAA y por último el puerto Link cambia de tamaño a uno más pequeño.

Game Boy Light

Esta revisión de la Game Boy es la más rara de ver, pues solo se distribuyó en Japón a partir de 1998. La Game Boy Light es muy similar a la Pocket en cuanto a las principales

[4] Autor: Evan-Amos.
Fuente: `https://commons.wikimedia.org/wiki/File:Game-Boy-FL.jpg`
[5] Autor: Evan-Amos.
Fuente:`https://commons.wikimedia.org/wiki/File:Game-Boy-Pocket-FL.jpg`



Figura 2.5.: Consola Game Boy Pocket.[5]

diferencias con la original, pero se hace destacar gracias a su pantalla iluminada y a su incremento de batería de hasta 20 horas sin la luz activada o de hasta 12 horas con la luz activa, usando 2 pilas AA para darle energía.



Figura 2.6.: Consola Game Boy Light.[6]

Game Boy Color

La Game Boy Color fue la última revisión que tuvo la Game Boy original y fue lanzada al mercado en el año 1998 para todo el mundo. La principal característica de esta es la

[6]Autor: Evan-Amos.
Fuente:`https://commons.wikimedia.org/wiki/File:Game-Boy-Light-FL.jpg`

capacidad de mostrar hasta 52 colores por pantalla de forma simultanea de una paleta de 32.768. Con esta mejora en la capacidad gráfica también vinieron mejoras en la capacidad de procesamiento, de hecho, se duplicó la frecuencia del procesador y se cuadruplicó la cantidad de memoria RAM respecto al resto de revisiones. Estos cambios provocaron que muchos de los juegos a partir de esta etapa, solo fueran compatibles con esta nueva versión de la consola.



Figura 2.7.: Consola Game Boy Colorl.[7]

2.2.3. Características Técnicas

Como ya se ha visto, la familia de consolas Game Boy ha pasado por varias revisiones desde su modelo base, por eso, para hacer un buen análisis técnico de la consola se deben tener en cuenta todas las versiones de la misma. La tabla 2.1 contiene las características técnicas de los tres principales modelos de Game Boy: La Game Boy original, la Game Boy Pocket y la Game Boy Color.

De entre todas las características de la Game Boy, la principal y más importante es su procesador, el Sharp LR35902.

[7] Autor: Evan-Amos.
Fuente:`https://commons.wikimedia.org/wiki/File:Nintendo-Game-Boy-Color-FL.jpg`

[8] Toda la información contenida en la tabla se ha obtenido de los datos técnicos oficiales de Nintendo [2]

Tabla 2.1.: Especificaciones técnicas de la consola Game Boy y sus revisiones.[8]

	Game Boy Original	**Game Boy Pocket**	**Game Boy Color**
CPU	Sharp LR35902 de 8 bits a 4.19 MHz	Sharp LR35902 de 8 bits a 4.19 MHz	Sharp LR35902 de 8 bits a 8 MHz
RAM	8 Kbytes	8 Kbytes	32 Kbytes
VRAM	8 Kbytes	8 Kbytes	16 Kbytes
ROM	De 32 Kbytes a 8 Mbytes	De 32 Kbytes a 8 Mbytes	De 32 Kbytes a 8 Mbytes
Dimensiones	148 x 90 x 32 mm	127.6 x 77.6 x 25.3 mm	133.5 x 78 x 27.4 mm
Peso	220 g	125 g	138 g
Consumo	70 - 80 mA hr. 4 pilas AA (1.5 V)	80 - 90 mA hr. 2 pilas AAA (1.5 V)	70 - 80 mA hr. 2 pilas AA (1.5 V)
Tiempo de Juego	Hasta 30 horas	Hasta 10 horas	Hasta 10 horas
Tipo de Pantalla	LCD	LCD	TFT
Tamaño de la Pantalla	4.7 x 4.3 cm	4.8 x 4.4 cm	4.4 x 4 cm
Resolución de la Pantalla	160 x 144 px	160 x 144 px	160 x 144 px
Colores	4 tonos de verde	4 tonos de gris	32.000 colores, 56 simultáneos

2.2.4. Procesador Sharp LR35902

En este apartado se va a analizar el funcionamiento del procesador de la Game Boy, el Sharp LR35902. El origen de este procesador está en el Zilog Z80, procesador usado en equipos como el MSX o el Amstrad CPC y en consolas como la Sega Master System o Sega Game Gear. A su vez, el Zilog Z80 está basado en la arquitectura del procesador Intel 8080, arquitectura que mejoraría añadiendo nuevos registros y una serie de instrucciones ampliada. Pese a estar fuertemente inspirado en el Z80, el Sharp LR35902 pierde los registros ampliados, pero conserva el juego de instrucciones ampliadas al que añade nuevas y aumenta la frecuencia de 2.5MHz a 4.19 y 8 MHz.

El principal objetivo del procesador es el de leer y ejecutar las instrucciones que permite su juego de instrucciones, y es mediante el correcto uso de estas instrucciones, que se pueden crear y ejecutar todos los juegos de la plataforma. Pero el primer paso para entender como funciona el procesador y sus instrucciones es entender qué son los registros y como se ven afectados por las instrucciones.

2.2.5. Registros

Un registro es un tipo de memoria de alta velocidad capaz de almacenar 8 bits de información. La Game Boy cuenta con 10 de estos registros que se emplean para realizar diversas operaciones en función del tipo de registro. Pese a solo poder almacenar 8 bits de información, ciertos registros de la Game Boy pueden unirse en pares para realizar operaciones de 16 bits.

El primero de los registros que se van a tratar es el registro A o acumulador. Este es uno de los más importantes pues es sobre el que se realizan la gran mayoría de operaciones aritméticas, almacenando el resultado de las mismas. Otra característica única a este registro es que es el único registro de 8 bits capaz de leer y escribir directamente en memoria sin necesitar registros auxiliares.

El siguiente de los registros es el F o flags. Este se encarga de almacenar los distintos flags que surgen tras cada instrucción. Cada instrucción ejecutada puede modificar desde 0 hasta 4 flags que se almacenan en 4 de los 8 bits disponibles del registro, obteniendo el valor 0 cuando no están activos y el valor 1 cuando se activan. Cada flag indica una característica sobre el resultado de una instrucción, y estos son:

- Flag de Acarreo (C). Se activa cuando en una operación aritmética hay acarreo. Se reinicia si la operación aritmética no produce acarreo.
- Flag de Medio Acarreo (H). Se activa cuando hay un acarreo en el bit 3 tras una operación aritmética. Se reinicia tras una operación aritmética que no produce acarreo en el bit 3. Su uso más común es en operaciones DAA usadas para representar valores decimales.
- Flag de Substracción (N). Se activa cuando la instrucción es una resta o substracción. Se reinicia cuando la instrucción es una suma o adición.
- Flag Cero (Z). Se activa cuando una operación da como resultado 0. Se reinicia cuando la operación resulta en un número distinto de 0.

Los registros B, C, D y E son registros que pueden usarse para almacenar valores de 8 bits o se pueden combinar en los pares BC y DE para poder almacenarlos de 16 bits. Los valores almacenados en estos registros pueden utilizarse como contadores o para realizar operaciones aritméticas junto al registro A y, al usarse en parejas, se suelen usar para almacenar direcciones de memoria o contadores de más de 8 bits.

Los registros H y L son muy similares a los anteriores, cada uno puede almacenar 8 bits de información, pero se pueden juntar para formar el registro HL capaz de almacenar hasta 16 bits. El principal uso de estos registros es el de almacenar direcciones de memoria, pues hay instrucciones que facilitan al registro HL el moverse por la memoria. Al igual que el registro A, el registro HL se puede usar para realizar operaciones aritméticas de 16 bits con registros como el BC o el DE.

Los dos últimos registros restantes son de 16 bits y se denominan como el registro SP (Stack Pointer) y el registro PC (Program Counter). El primero de estos se encarga de guardar el último valor de la pila de llamadas. Esta pila puede variar su valor en función de la instrucción que se ejecute, y se usa habitualmente para hacer llamadas a procedimientos o almacenar valores de los registros que se quieran recuperar. El PC se encarga de almacenar la dirección de la próxima instrucción que va a ser ejecutada.

Ahora bien, los registros del procesador no están limitados a lo que se ha explicado hasta ahora, lo único que limita lo que cada registro puede hacer son las instrucciones del procesador y lo que el programador decida hacer con ellas. Por eso mismo, el siguiente apartado va a estar dedicado a las diferentes instrucciones que pueden ser ejecutadas por el procesador.

2.2.6. Instrucciones

A continuación se van a ver los distintos grupos en los que se pueden diferenciar las instrucciones que es capaz de ejecutar el procesador de la Game Boy, y estos son:

- Instrucciones de carga de 8 y 16 bits. Estas instrucciones se centran en cargar 8 o 16 bits de información a los diferentes registros o direcciones de memoria. Unos ejemplos de estas instrucciones son: LD, PUSH, POP, LDI o LDD.

- Instrucciones aritméticas de 8 bits. Estas instrucciones son las responsables de realizar cálculos aritméticos tales como adición, substracción o comparación. Todas estas instrucciones requieren del uso del registro A, y algunos de los ejemplos más usados son: ADD, SUB, INC, DEC o CP.

- Instrucciones aritméticas de 16 bits. Al igual que en el caso anterior estas instrucciones realizan operaciones aritméticas, con la diferencia de que lo hacen sobre el registro HL de 16 bits y sin la instrucción de comparación. Unos ejemplos de estas instrucciones son: ADD, SUB, INC o DEC.

- Instrucciones de operaciones lógicas. Estas son instrucciones de 8 bits que operan sobre el registro A, realizando, como su nombre indica, operaciones lógicas a nivel de bit. Algunos ejemplos de este tipo de instrucciones son: AND, OR o XOR.

- Instrucciones de control de la CPU. Este tipo de instrucciones permiten modificar en cierta medida el comportamiento de la CPU. Algunos ejemplos son: DI, EI, HALT o NOP.

- Instrucciones de rotación y cambios. Estas instrucciones operan directamente sobre los registros y permiten mover y rotar los bits de los valores que hay dentro de los mismos. Uno de los usos más extendido de este tipo de instrucciones es que permiten multiplicar y dividir entre 2. Algunos ejemplos son: RLA, RLC, SLA o SRL.

- Instrucciones a nivel de bit. Estas instrucciones permiten comprobar y modificar el valor de un bit en concreto dentro de un registro de 8 bits. Estas instrucciones son: BIT, SET y RES.

- Instrucciones de saltos y llamadas. Estas, son las instrucciones que permiten moverse por el código de una manera no lineal. Su uso principal es el de saltarse partes de código que no se quiere ejecutar o el de llamar a procedimientos o funciones. Estas intrucciones son: JP, JR, CALL, RET y RETI.

Cada instrucción mencionada tiene asignado un opcode, que viene a representar su codificación en hexadecimal, y un número de ciclos que tarda en ejecutarse. Esto es relevante pues debido a la limitada capacidad de la consola, el conocer cuanto ocupa una instrucción y cuanto tarda en completarse es muy importante a la hora de programar un juego de manera eficiente.

2.2.7. Mapa de Memoria

La Game Boy cuenta con direcciones de memoria de 16 bits, lo que le permite acceder a 65.535 direcciones de memoria distintas, representadas desde la dirección 0000_{hex} hasta la FFFF_{hex}. Estas direcciones de memoria a su vez, están divididas en bloques que agrupan diversas funciones, tal y como se puede ver en la tabla [2.2]. Estas divisiones son absolutas, es decir, no pueden cambiarse de sitio y, algunas de estas tienen condiciones para poder leer o escribir de ellas como la VRAM o el OAM.

Pese a que la memoria parece muy limitada existen los bancos de memoria, que permiten expandir ciertos bloques de memoria, como el de ROM o el de RAM externa, a capacidades que exceden las establecidas. Estos bancos no añaden memoria de forma directa, en su lugar, permiten que una misma dirección de memoria pueda acceder a los diferentes grupos de datos llamados bancos. Estos bancos pueden intercambiarse entre ellos mediante el uso mappers incluidos en el cartucho.

Por último mencionar que muchos cartuchos incluían una pila interna, esto se hacia para que el estado de la RAM externa no se eliminara una vez se apagara la consola. Gracias a este proceso, se consiguió una forma efectiva de guardar partida excluyendo a las anteriormente usadas contraseñas.

Tabla 2.2.: Distribución del mapa de memoria de la Game Boy.

Direcciones de memoria	**Contenido**
0000_{hex} - $3FFF_{hex}$	16 KB de ROM, Banco 0 En el cartucho
4000_{hex} - $7FFF_{hex}$	16 KB de ROM, Banco 1 - N Intercambiable En el cartucho
8000_{hex} - $9FFF_{hex}$	8 KB de VRAM
$A000_{hex}$ - $BFFF_{hex}$	8KB de RAM externa Intercambiable En el cartucho
$C000_{hex}$ - $CFFF_{hex}$	4KB de RAM, Banco 0
$D000_{hex}$ - $DFFF_{hex}$	4KB de RAM, Banco 1 - 7 Intercambiable en modo CGB
$E000_{hex}$ - $FDFF_{hex}$	Copia de C000 a DDFF No se suele usar
$FE00_{hex}$ - $FE9F_{hex}$	Atributos de los sprites, OAM
$FEA0_{hex}$ - $FEFF_{hex}$	No usable
$FF00_{hex}$ - $FF7F_{hex}$	Puertos Hardware
$FF80_{hex}$ - $FFFE_{hex}$	HRAM, Ram Alta
$FFFF_{hex}$ - $FFFF_{hex}$	Registro de interrupciones

2.2.8. Memoria ROM

La memoria ROM es una de las más importantes a la hora de programar, pues es donde se va a almacenar inicialmente todo el código del juego. Esta memoria también contiene una cabecera situada al comienzo de la misma donde se indican diversas características necesarias para el correcto funcionamiento del juego. En la tabla [2.3] se pueden observar todos los componentes de la memoria ROM y la función que desempeñan.

Tabla 2.3.: Mapeado de la memoria ROM

Direcciones de Memoria	**Contenido**
0000_{hex} - $003F_{hex}$	Llamadas RST
0040_{hex} - 0047_{hex}	Interrupción Vblank.
0048_{hex} - $004F_{hex}$	Interrupción LCD-Stat.
0050_{hex} - 0057_{hex}	Interrupción temporizador.
0058_{hex} - $005F_{hex}$	Interrupción serial.
0060_{hex} - 0067_{hex}	Interrupción joypad.
0068_{hex} - $00FF_{hex}$	Sin uso.
0100_{hex} - 0103_{hex}	Punto de entrada.
0104_{hex} - 0133_{hex}	Logo de Nintendo.
0134_{hex} - 0142_{hex}	Nombre del juego.
0143_{hex}	Flag de la Game Boy Color. 80_{hex} = GB+CGB, $C0_{hex}$ = CGB.
0144_{hex} - 0145_{hex}	Código de manufactura.
0146_{hex}	Flag de la Super Game Boy.
0147_{hex}	Tipo de cartucho.
0148_{hex}	Tamaño de la ROM. 0 = 32 KiB. 1 = 64 KiB, 2 = 128KiB, etc.
0149_{hex}	Tamaño de la RAM. 0 = Nada. 1 = 2 KiB, 2 = 8KiB, 3 = 32 KiB
$014A_{hex}$	Código regional. 0 = JPN, 1 = EU/US
$014B_{hex}$	Código de licencia antiguo.
$014C_{hex}$	Version de la ROM.
$014D_{hex}$	Checksum de cabecera.
$014E_{hex}$ - $014F_{hex}$	Checksum global.
$014F_{hex}$ - $3FFF_{hex}$	Banco 0 de la ROM
4000_{hex} - $7FFF_{hex}$	Banco intercambiable de la ROM

2.2.9. Memoria de Vídeo

Este es el último apartado relacionado con las características técnicas de la Game Boy y sus revisiones, y es donde se va a analizar la estructura y el funcionamiento tanto de la memoria de vídeo como del OAM.

La memoria de vídeo de la Game Boy se puede dividir en 3 grupos principales, tal y como se puede ver en la tabla [2.4]. Cada uno de estos grupos desempeña una función distinta a la hora de mostrar imágenes por pantalla. Pero antes de analizar cada uno de estos individualmente se deben entender las diferentes partes del dibujado por pantalla.

Tabla 2.4.: Distribución de la memoria de vídeo.

Dirección de memoria	**Contenido**
8000_{hex} - $97FF_{hex}$	Tilesets
9800_{hex} - $9FFF_{hex}$	Tilemaps
$FE00_{hex}$ - $FEFF_{hex}$	OAM

Dibujado de la Game Boy

Pese a que la Game Boy cuenta con una pantalla LCD capaz de dibujar 160 x 144 píxeles, internamente no puede dibujar píxeles sueltos. En su lugar, la Game Boy está basada en un sistema de tiles de 8 x 8 px. A la hora de dibujar, la Game Boy cuenta con un fondo de 32 x 32 tiles, de los cuales solo se pueden ver una porción de 20 x 18 px llamada vista. Esta vista puede moverse libremente por el fondo empleando los registros ScrollX y ScrollY, con la peculiaridad de que si sobrepasa uno de los límites, empezará a mostrar los tiles del límite opuesto. Por último están los sprites, estos tienen la peculiaridad de poder dibujarse sin estar regidos por una cuadrícula y su posición es relativa a la vista.

Al igual que hay registros para posicionar la vista dentro del fondo, existen otros registros que ayudan a controlar el sistema de vídeo de la pantalla. Estos registros se desarrollarán en la sección de desarrollo [6].

Una vez se tiene clara la forma que tiene la Game Boy de representar imágenes por pantalla, se va a proceder a explicar cada grupo en el que se puede dividir la memoria de vídeo, empezando por el tileset.

Tileset

El tileset es una sección de la memoria de video comprendida desde la dirección 8000_{hex} hasta la $97FF_{hex}$. En esta sección es donde se cargan inicialmente todos los tiles que van a ser usados para representar el fondo o los propios sprites. Cada tile codificado ocupa un total de 16 bytes por lo que se pueden cargar un total de 384 tiles de forma simultanea.

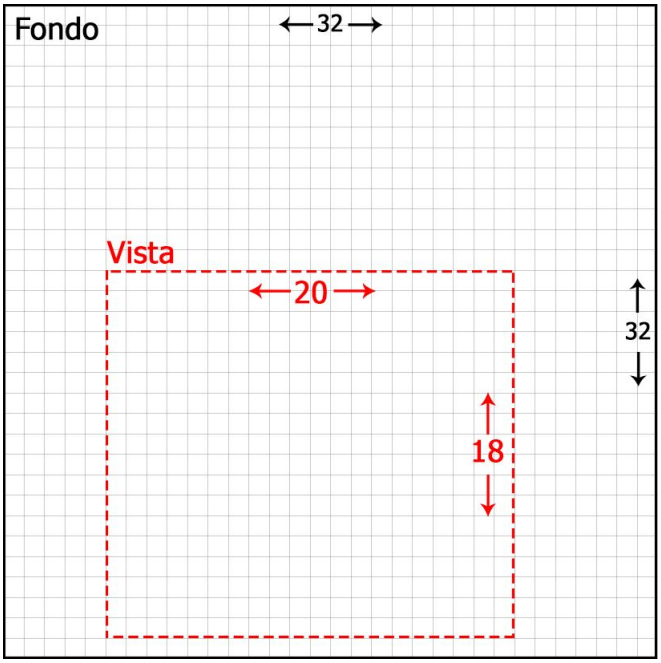


Figura 2.8.: Esquema de la relación entre el fondo y la vista.

Tilemaps

El tilemap es lo que antes se ha llamado como fondo a la hora de explicar el funcionamiento del dibujado por pantalla. En el caso de la Game Boy existen dos tilemaps llamados BG map, que ocupa de la dirección 9800_{hex} a la $9BFF_{\text{hex}}$ y Window, que ocupa de la dirección $9C00_{\text{hex}}$ a la $9FFF_{\text{hex}}$. Normalmente el uso de los tilemaps es el siguiente, el BG map se usa para representar el fondo de la escena mientras que la Window se usa para solapar elementos como la interfaz de usuario.

Cada byte de estas direcciones de memoria hace referencia a la posición de uno de los tiles definidos en el tileset, de los cuales solo se puede acceder a un total de 256 tiles de los 384 que se pueden declarar. Esto permite dejar los 128 tiles restantes para otros usos como la respresentación de los sprites.

Sprites

Por último quedan los sprites. Estos se localizan en una zona de memoria separada del resto de elementos de la memoria de vídeo. Esta zona de memoria se situa entre las direcciones $FE00_{\text{hex}}$ y la $FEFF_{\text{hex}}$ y se llama OAM o Object Attribute Memory.

La Game Boy puede almacenar y mostrar hasta 40 sprites por pantalla, con la limitación de un máximo de 10 sprites por línea de dibujado. Esta limitación viene dada por el hardware, pues si se excede el máximo de sprites por línea, se producirá un efecto de parpadeo en los mismos.

Cada sprite ocupa 4 bytes de memoria. Los dos primeros bytes hacen referencia a la posición Y y X respecto a la vista. El tercer byte indica el número del tile que se va a usar como imagen, de la misma forma que se hace para los tilemaps. El último byte es el de atributos, donde cada bit del mismo indica una característica del sprite:

- Bits 0, 1 y 2: Indican la paleta que se va a usar en el modo de Game Boy Color.
- Bit 3: Indica el banco de tiles que se va a usar. Solo se usa en modo Game Boy Color.
- Bit 4: Indica la paleta que se va a usar para la Game Boy original.
- Bit 5: Activa el volteado Horizontal.
- Bit 6: Activa el volteado Vertical.
- Bit 7: Indica la prioridad del sprite respecto al fondo.

Ahora que se han explicado las diferentes partes de la memoria de vídeo, cabe destacar la imagen [2.9] donde se muestra la relación entre las mismas.

2.3. Juegos que han influenciado al proyecto

Previo al desarrollo de este proyecto han habido otros proyectos que han servido como base para modelar las mecánicas y el estilo del mismo. Todos los juegos que se van a

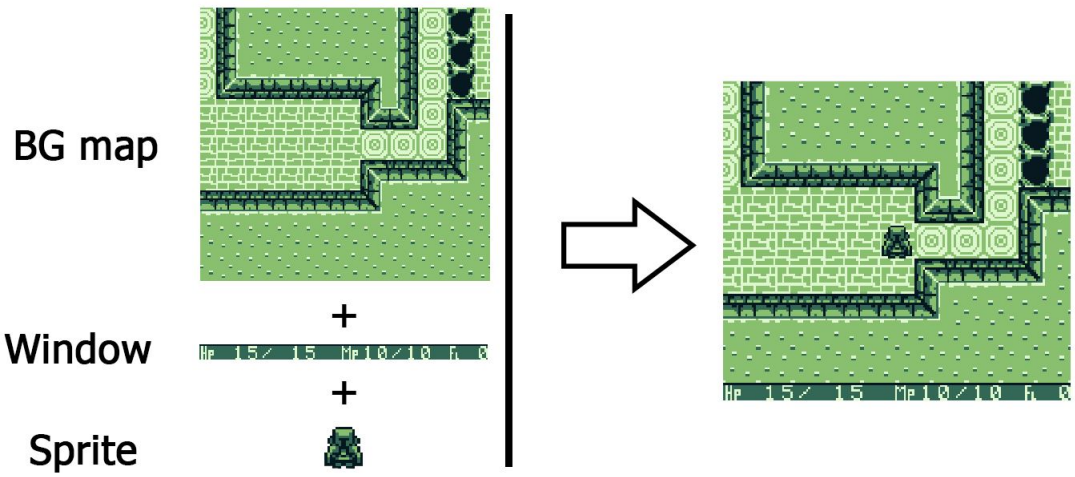


Figura 2.9.: Esquema que muestra el funcionamiento de las diferentes partes de la memoria de vídeo.

mencionar a continuación han aportado en al menos uno de los siguientes aspectos: a nivel mecánico, a nivel técnico y a nivel estético.

2.3.1. Rogue

Rogue es un juego de exploración de mazmorras desarrollado por Michael Toy, Glenn Wichman y Ken Arnold, que fue lanzado al mercado en 1980 y adaptado para multitud de sistemas como Amiga, Macintosh o Amstrad CPC entre otros. La premisa de este juego es la de ponerse en la piel de un aventurero que se decide a adentrarse en una gran mazmorra. El objetivo es ir descendiendo por la mazmorra para recuperar el Amuleto de Yendor y volver hasta la superficie, con la peculiaridad de que cada piso de la mazmorra se genera de forma procedimental, generando salas pasillos, objetos, y enemigos de forma diferente cada vez que el jugador avanza de piso. Si el jugador pierde, debe empezar desde el principio de la mazmorra perdiendo todo el progreso y equipo recogido.

La jugabilidad es por turnos, donde en cada uno de estos el jugador realiza una acción, ya sea moverse, atacar o usar un objeto y acto seguido el resto de entidades realizan la suya propia. El juego también incorpora un sistema de inventario para usar objetos y equipo, un sistema de estadísticas y un sistema de niveles.

Gráficamente, el juego se representa mediante caracteres ASCII, donde cada uno de estos representa un elemento del juego, como por ejemplo la letra S representa una serpiente.

[9] Autor: Artoftransformation.
Fuente:`https://commons.wikimedia.org/wiki/File:Rogue_Screen_Shot_CAR.PNG`



Figura 2.10.: captura del vieojuego Rogue.[9]

Rogue fue el que impulsó el género actualmente conocido como roguelike, un género que coge las ideas del mapa generado procedimentalmente, el reinicio del progreso al morir y la exploración de mazmorras, y las aplica de las formas más diversas posibles.

Por último, este y el próximo juego que se va a describir son los que han servido como base para la realización de todas las mecánicas jugables de este proyecto, desde la generación de mapas, hasta el combate y el sistema de turnos.

2.3.2. Shiren the Wanderer GB

Shiren the Wanderer GB también conocido como Furai no Shiren GB es un juego que pertenece al género del roguelike desarrollado por Chunsoft y lanzado al mercado en 1996. Este juego es la primera entrega adaptada a la Game Boy de la saga Mistery Dungeon. Esta saga es conocida por adaptar la jugabilidad descrita en el juego Rogue añadiéndole una capa de narrativa lo que convierte la experiencia de recorrer mazmorras en un instrumento para avanzar la trama.

Al igual que en Rogue, las mazmorras se generan de forma procedimental, el movimiento es por turnos y hay un sistema de objetos e inventario. A todo esto, el juego añade localizaciones diseñadas a mano donde avanzar la trama y personajes no jugables con los que interactuar fuera del combate.

Gráficamente, el juego emplea sprites con las limitaciones de resolución y color que vienen definidas por la Game Boy, aunque cuenta con una paleta de colores para adaptarse al hardware de Game Boy Color.

Este juego ha sido elegido debido a las características técnicas que se pueden observar del mismo. Puesto que es un juego de Game Boy, se puede ver el comportamiento del juego a nivel técnico y adaptarlo al proyecto, lo que ha ayudado a la hora de realizar el scroll de la pantalla, la carga de mapas, el sistema del HUD y la IA de los enemigos.



Figura 2.11.: captura del vieojuego Shiren the Wanderer GB.

2.3.3. The Legend of Zelda: Link's Awakening

El último juego en el que se ha basado el proyecto es el The Legendo of Zelda: Link's Awakening. Este es un juego de acción aventura desarrollado por Nintendo para la Game Boy y lanzado en 1993. El objetivo de este juego es el de encontrar 8 instrumentos esparcidos en diferentes mazmorras alrededor el mundo con el fin de despertar al Pez del Viento. Cada mazmorra plantea una serie de retos que se pueden solucionar empleando alguno de los objetos que el protagonista tiene a su disposición.

El juego se representa en vista cenital y ofrece un escenario donde el jugador puede moverse libremente, interactuar con PNJ y combatir contra enemigos. También cuenta con una serie de objetos que ofrecen mejoras tales como la capacidad de bloquear ataques o saltar pequeños huecos del suelo.

Gráficamente el juego cuenta con sprites limitados en resolución y a 4 tonos de gris, aunque más adelante sacarían una versión para la Game Boy Color que añadiría nuevas paletas de colores y algunas mejoras técnicas.

El principal motivo para usar este juego como base ha sido el apartado estético de las mazmorras y de los diferentes elementos que las pueblan.



Figura 2.12.: captura del vieojuego The Legend of Zelda: Link's Awakening.

3. Objetivos

La realización de este proyecto trae consigo una serie de objetivos, los cuales explicaré brevemente a continuación para su futuro desarrollo:

- Elaborar un videojuego del género roguelike.
- Diseñar el concepto de un videojuego a partir de una base establecida, añadiendo y modificando mecánicas para obtener un resultado único.
- Implementar las mecánicas del jugador, tales como el movimiento, combate, gestión del inventario y uso de objeto
- Desarrollar un generador de mapas procedimental de manera que cada mazmorra cuente con una distribución única de enemigos, objetos y salas.
- Aprender el funcionamiento interno de la consola Game Boy así como sus capacidades y arquitectura.
- Conocer el lenguaje ensamblador así como las múltiples herramientas disponibles para el desarrollo de un videojuego para la Game Boy.
- Elaborar material de referencia para otros desarrolladores interesados en el desarrollo para la Game Boy.

4. Metodología

Para el desarrollo de este proyecto se ha decidido usar una metodología Iterativa por la facilidad y el control que proporciona a la hora de escalar el proyecto. A continuación se procederá a describir el funcionamiento de este tipo de metodología.

La metodología iterativa se plantea en diversos bloques temporales llamados iteraciones. En cada una de estas iteraciones se genera un prototipo del proyecto, que permite ver las funcionalidades básicas que se desean implementar sin necesidad de incorporar toda la lógica del propio proyecto. Este prototipo permite evaluar si las funcionalidades implementadas son validas o se deben modificar. Poco a poco conforme se van desarrollando las iteraciones, estas se van añadiendo al proyecto final para ir aumentando su funcionalidad.

Como complemento a la metodología iterativa, se han descrito las diferentes tareas necesarias para el desarrollo del videojuego con ordenes de prioridad para orientar el avance del proyecto. Estas mismas tareas también sirven para comprobar el estado del proyecto en cierto punto.

4.1. Gestión del proyecto

Una vez explicada la metodología que se va a usar, se van a comentar las diferentes herramientas que se han usado para gestionar el proyecto y sus versiones.

4.1.1. Trello

Trello es un servicio web gratuito que permite gestionar proyectos mediante tarjetas. En estas tarjetas se pueden definir tareas y sus condiciones para ser completadas. Esta herramienta permite clasificar las diferentes tareas encontradas en el proyecto en diferentes grupos personalizados como 'pendiente' o 'completado', y dentro de cada grupo se pueden organizar en orden de prioridad, siendo las tareas más prioritarias las que aparecen primero. En el caso de este proyecto se han creado los grupos de pendiente, en cola, en desarrollo, completado y un grupo por cada hito de desarrollo.

4.1.2. GitHub

Git es el programa de gestión de versiones que se va a usar durante el desarrollo del proyecto, y para almacenar el repositorio generado se va a hacer uso del sitio web GitHub. El uso de la plataforma de GitHub es muy relevante a la hora de crear un proyecto ya sea individual o en grupo, dado que permite realizar un control de las diferentes versiones por las que ha pasado el proyecto, y el caso de que algo no funcione, siempre se puede



Figura 4.1.: captura del tablero de Trello.

volver a una versión anterior. La otra gran utilidad de GitHub es que sirve como copia de seguridad en caso de que el equipo de trabajo deje de funcionar.

En el caso de este proyecto se han creado dos ramas una de desarrollo y una master. En la rama de desarrollo es donde se van implementando las diferentes funcionalidades del proyecto, mientras que a la rama master solo se sube una versión estable del proyecto.

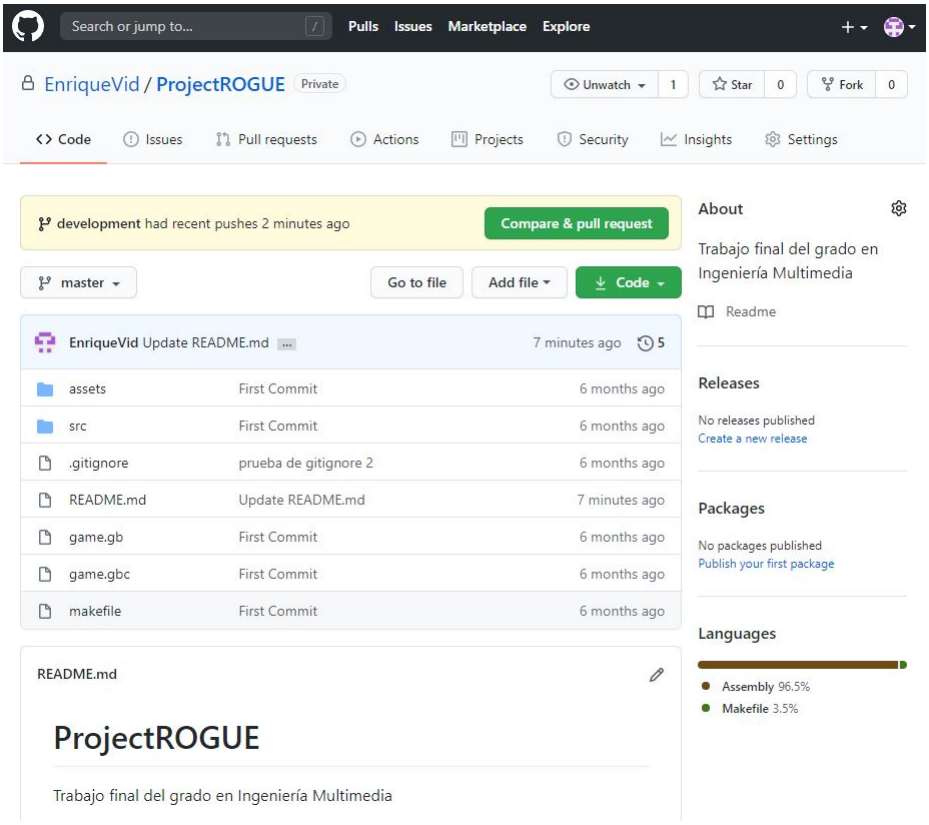


Figura 4.2.: captura de la página de GitHub.

5. Game Design Document

En este apartado se van a describir todos los aspectos referentes a cómo debería quedar el juego en su estado final de desarrollo. Hay que tener en cuenta que, debido al ámbito del proyecto, no todos los apartados van a ser completados.

5.1. Resumen

5.1.1. Introducción

Project Rogue es un juego de rol inspirado en el clásico Rogue de 1980 así como en la saga Mistery Dungeon. Más en concreto, está principalmente basado en el juego Shiren the Wanderer GB para la Game Boy.

Este juego está desarrollado con fines académicos para la videoconsola Game Boy de Nintendo.

El juego consiste en explorar una gran mazmorra dividida en pisos que, cada vez que visitas, generan de manera procedimental la estructura del mapa y la disposición de enemigos y objetos. Todo esto para que la experiencia de juego sea variada y única en cada intento.

La mazmorra se visualiza en vista cenital y están regidas por una cuadrícula. En esta, se encontrarán objetos, que podrás guardar en el inventario, usar o equipar, así como enemigos que serán el principal bloque a superar.

El objetivo es alcanzar las escaleras de cada piso para avanzar al siguiente. Cada cierto número de pisos habrá un combate contra un enemigo más poderoso y tras derrotarlo, guardará un punto de control así como se dará la opción de regresar a la base.

La mencionada base es un lugar apartado de la acción donde comprar objetos, subir de nivel y elegir el equipo para adentrarse de nuevo en la mazmorra. A esta base se accede cuando mueres, tras derrotar a un enemigo poderoso (se te da la opción) y mediante el uso de objetos.

La acción se realiza por turnos, en cada turno puedes elegir moverte, atacar o usar algún objeto. Una vez hecha la acción, los enemigos efectuarán la suya y te devolverán el control.

5.1.2. Características clave

Cada piso de la mazmorra, así como sus objetos, enemigos y salida se generan de forma procedimental siguiendo un algoritmo de generación basado en salas, que contienen los objetos y la salida, y pasillos para unir las diferentes salas.

Los enemigos reaparecen de forma constante por el mapa hasta un máximo definido. La reaparición no es instantánea, cuando un enemigo muere se establece un número de turnos para que aparezca otro en la mazmorra.

El personaje cuenta con un sistema de niveles que mejora sus estadísticas base. Algunos enemigos eliminados y algunos objetos otorgan dinero, que se puede gastar en comprar objetos o subir de nivel en la base. La elección no siempre estará clara pues en ciertas situaciones se deberá priorizar el comprar objetos frente a subir de nivel para afrontar las situaciones más complicadas.

El protagonista puede equiparse con un arma principal, un escudo, así como con un objeto asignable, que puede ser un objeto consumible o un objeto mágico. Al usar el objeto asignado este realizará la acción correspondiente al mismo.

Al contrario que en Rogue, que cada vez que mueres se reinicia el juego, en Project Rogue regresas a la base donde puedes gestionar los objetos y subir de nivel. A la hora de morir, se perderán la mitad de los objetos consumibles y se dejará la mitad del dinero obtenido en el piso de la muerte. Se podrá recuperar el dinero si se vuelve al piso donde se murió y encuentra la bolsa con el mismo.

En la mazmorra hay checkpoints cada x niveles después de un combate contra un enemigo superior. Estos checkpoints son accesibles desde la base para retomar la aventura.

5.2. Mecánicas Básicas

5.2.1. Jugador:

Movimiento:

- El movimiento se realiza sobre una cuadrícula y el jugador puede moverse en 8 direcciones. El jugador se mueve por casillas de 16 x 16 px y no puede quedar entre dos de las casillas. Una vez se mueve el jugador, se mueven el resto de enemigos. Si el jugador no se mueve, los enemigos no realizan ninguna acción.

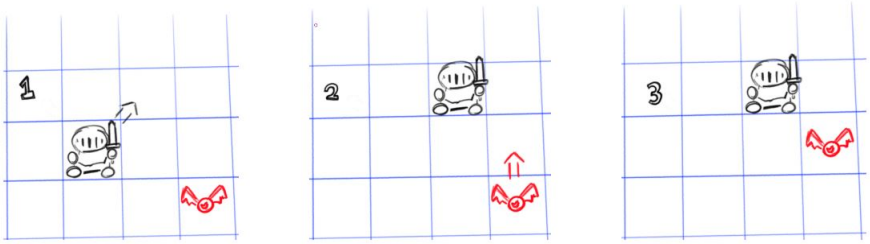


Figura 5.1.: Boceto que representa el movimiento del jugador y los enemigos.

- Mantener el botón 'B' permite orientar al jugador en cualquiera de las 8 direcciones sin moverse del sitio. Esto es útil si estás rodeado de enemigos y se quiere atacar

a uno en concreto.

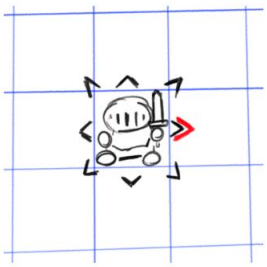


Figura 5.2.: Boceto que representa la elección de dirección del jugador.

- El movimiento queda restringido si en la dirección a la que te quieres mover hay una pared, un elemento sólido o un PNJ ya sea amigo o enemigo.

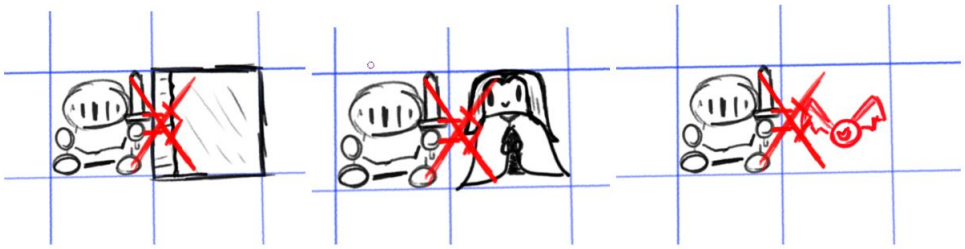


Figura 5.3.: Boceto que representa las colisiones del jugador.

- Moverse sobre un objeto o la salida provoca, en el caso del objeto su recogida automática (si hay espacio en el inventario) y, en el caso de la salida, que se interactúe con con la misma. (Mecánicas de objetos y salida explicadas en la sección de objetos y mazmorra).

Combate:

- Si el jugador está frente al enemigo y ataca, el enemigo recibe daño. Acto seguido el enemigo realiza su acción.
- Tras atacar el jugador, si hay varios enemigos, atacarán todos ellos uno detrás del otro. Si un enemigo no puede atacar optará por moverse.
- Se puede atacar al aire para pasar un turno sin moverte.
- El daño del combate se calcula usando el siguiente algoritmo, donde A equivale al daño del atacante y D es la defensa del atacado:

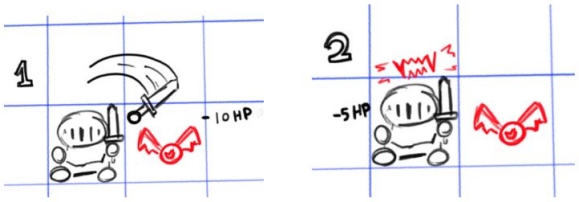


Figura 5.4.: Boceto que representa la acción de atacar a un enemigo.

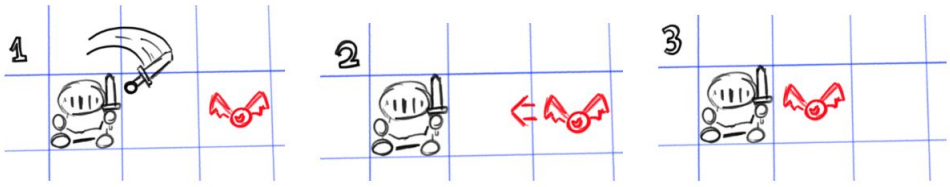


Figura 5.5.: Boceto que representa la acción de pasar turno atacando al aire.

$$D = A - \lfloor D/2 \rfloor \tag{5.1}$$

$$R = \begin{cases} 1 & \text{si } D < 1 \\ D & \text{si } D \geq 1 \end{cases}$$

Magia:

- El personaje puede llevar asignado 1 objeto, que puede ser de hechizo.
- Cada hechizo tiene su propio daño asociado e ignora la estadística de fuerza del personaje que lo usa.
- Los hechizos tienen efectos y alcances varios, por ejemplo la "Espada de trueno" tiene el mismo alcance que el ataque normal, sin embargo la "bola de fuego" es un proyectil que avanza hasta colisionar con algo.
- El uso de hechizos consume puntos de Magia y cada hechizo tiene un consumo distinto.

Estados Alterados:

- Durante la aventura el personaje puede contraer diferentes estados alterados que le afectarán de diferentes formas.

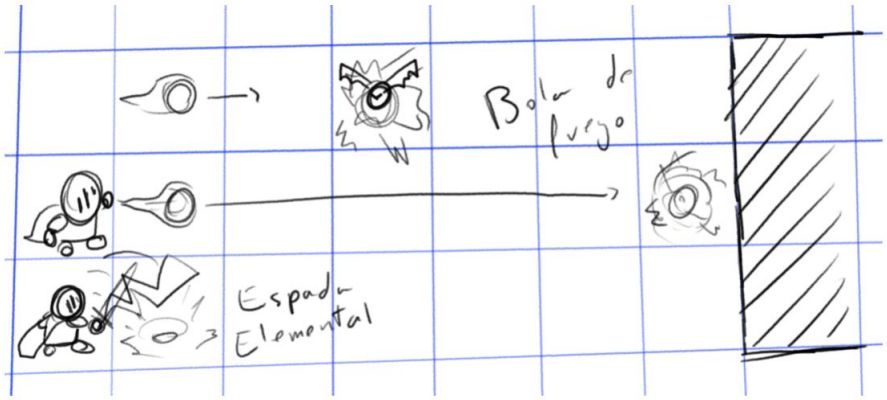


Figura 5.6.: Boceto que representa los diferentes tipos de ataques mágicos.

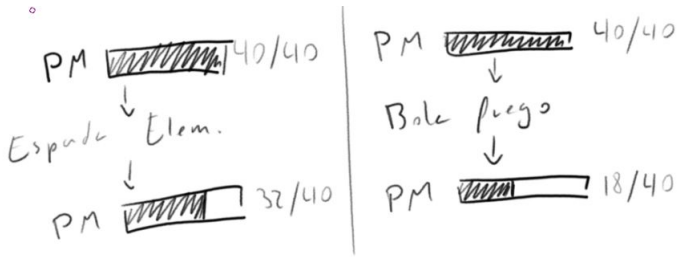


Figura 5.7.: Boceto que representa el consumo de puntos de magia.

- Los estados alterados pueden ser positivos o negativos y se anulan con objetos, esperando turnos o cambiando de piso en la mazmorra.
- Veneno: Merma 5PV cada 5 pasos. Puede ser curado usando un antídoto o cambiando de piso de la mazmorra.



Figura 5.8.: Boceto que representa el estado alterado veneno.

- Parálisis: Impide el movimiento, pero permite atacar. Dura una cantidad de 3 a 5 turnos.



Figura 5.9.: Boceto que representa el estado alterado parálisis.

Muerte:

- Si durante el combate, los puntos de vida llegan a cero, se pierde la partida y se regresa de forma automática a la base.
- Cuando el personaje principal muere, el 50 % de los objetos recogidos que no estén equipados se pierden.
- Cuando el personaje principal muere, el 50 % del dinero acumulado se queda en el piso de la muerte, siendo recuperable si se vuelve al piso y se encuentra. Pero si se muere una segunda vez antes de recuperar el dinero perdido, este se perderá para siempre.

Estadísticas:



Figura 5.10.: Boceto que representa la muerte del jugador.

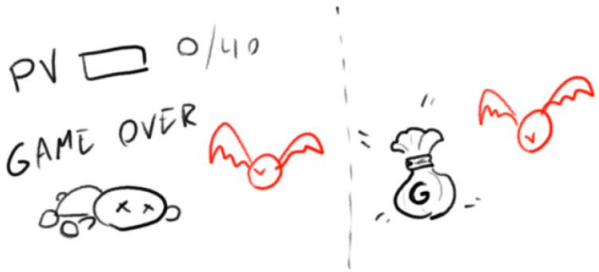


Figura 5.11.: Boceto que representa los objetos dejados por el jugador.

- Tanto el personaje como los enemigos cuentan con estadísticas que determinan su capacidad tanto ofensiva como defensiva. En el caso del protagonista, se pueden aumentar de forma permanente subiendo de nivel o de forma temporal usando equipo o mediante estados alterados.
- Vida: Si los puntos de vida llegan a cero, el personaje muere. Se pueden recuperar con objetos o caminando (1 PV cada 5 pasos)
- Fuerza: Cuanto mayor sea este valor más puntos de vida mermará del rival.
- Puntos de magia: Se consumen a la hora de realizar hechizos, si son menores a los necesarios para realizar un hechizo, éste no se podrá efectuar. Se pueden recuperar con objetos.
- Defensa: Cuanto mayor sea este valor, menos efecto tendrá el ataque enemigo.

Subida de nivel:

- En la base, se pueden hacer ofrendas monetarias en un altar para subir de nivel. Es decir, se gasta la moneda del juego para incrementar el nivel del personaje principal.
- Cada subida de nivel permite aumentar una estadística del personaje en una unidad a voluntad, permitiendo una mayor libertad a la hora de personalizar al personaje.
- Cada vez que se sube de nivel el precio para subir al siguiente aumenta. Esto fomenta que el jugador avance por la mazmorra y no se quede en los niveles más bajos.

Inventario:

- El jugador cuenta con un inventario limitado a 20 huecos para almacenar los objetos mientras recorre la mazmorra.
- Este inventario es accesible desde el menú de pausa y permite tanto asignar equipo nuevo cómo usar algún objeto consumible para restaurar salud o regresar a la base.
- Tanto equipar un objeto como usarlo cuentan como una acción para pasar el turno.
- Los objetos en el inventario se pueden: usar, equipar/quitar, desechar y asignar.

5.2.2. Enemigos

Movimiento:

- Los enemigos se mueven una vez el jugador ha realizado su acción (moverse, atacar, usar un objeto).
- Se pueden mover en una de las 8 posibles direcciones, de la misma forma que lo hace el jugador.

- Los enemigos no se pueden mover a una posición con un elemento sólido o en la que haya otro PNJ.
- Los enemigos intentarán dirigirse hacia el jugador, siempre que no haya algo que se lo impida.
- Algunos enemigos aparecen como inactivos. No hacen nada hasta que el jugador se posiciona a una casilla de éstos o les ataca.

Combate:

- Los enemigos atacan una vez el jugador ha atacado. Lo hacen por turnos, un enemigo detrás de otro. El orden de ataque es el orden en el que se han generado los enemigos.
- Si un enemigo no puede atacar optará por moverse en dirección al jugador.
- Cada enemigo ataca de forma diferente, unos atacan cuerpo a cuerpo, otros a distancia y los hay que intentan provocar estados alterados. Todo depende del tipo de enemigo.

Estadísticas:

- Los enemigos al igual que el jugador cuentan con estadísticas que definen su nivel de peligro. Tienen las mismas estadísticas que el jugador excepto los puntos de magia pues de usar hechizos como ataque, lo podrán hacer de forma infinita.
- Vida: Si los puntos de vida llegan a cero, el enemigo muere.
- Fuerza: Cuanto mayor sea este valor más puntos de vida mermará del rival.
- Defensa: Cuanto mayor sea este valor, menos efecto tendrá el ataque enemigo.

Jefes:

- Los enemigos superiores o jefes se comportan de la misma forma que el resto de enemigos. Lo que los diferencia son sus estadísticas, pues los jefes tendrán mucha más vida y obligarán al jugador a emplear todos los objetos y magias que tenga a su disposición.
- Los jefes derrotados pueden ser combatidos de nuevo si desde la base se accede a la zona de la mazmorra que da paso al jefe que se busca.
- Cuando un jefe es derrotado, dejará caer un objeto de mayor valor así como dinero. Cada jefe tiene una 'pool' de objetos que puede dejar, entre ellos: armas, escudos, objetos mágicos o dinero.
- Los objetos dejados por un jefe son únicos, es decir, si te enfrentas al mismo jefe varias veces nunca te dará el mismo objeto 2 veces. Si la 'pool' de objetos se vacía, el jefe sólo dejará el dinero.

5.2.3. Objetos

Cuando caminas sobre un objeto este se añade automáticamente al inventario. En el caso de que no quede espacio en el inventario el objeto permanecerá en el suelo sin ser recogido.

Los objetos pueden ser de tres tipos.

Consumibles:

- Tienen un solo uso y aportan efectos variados durante el piso en el que estas. Tienen dos categorías: ofensivos y defensivos.
- Los defensivos pueden usarse para curar estados alterados, recuperar puntos de vida o magia, o para aplicar estados alterados positivos al jugador (subida de daño, mejora de defensa, etc)
- Los ofensivos se basan en aplicar estados alterados negativos al enemigo (daño, parálisis, veneno, etc).
- Los ofensivos pueden usarse como proyectil o cuerpo a cuerpo

Equipables:

- Se equipan al personaje y aportan subida a las características del mismo. Se dividen en armas y escudos.
- Pueden aportar mejoras a las estadísticas de ataque, defensa, vida o puntos de magia.
- Solo se puede tener un objeto equipable del mismo tipo equipado a la vez (ej: solo se puede tener un escudo equipado a la vez).

Mágicos:

- Pueden usarse siempre y cuando se tengan los Puntos de Magia necesarios. No se consumen al ser usados.
- Pueden asignarse para su rápido uso en combate.

5.2.4. Mazmorra

Generación:

- Cada piso de la mazmorra se genera de forma procedimental cada vez que lo visitas, esto provoca que no se repita el mismo piso dos veces seguidas.
- Cada piso de la mazmorra puede generar de 3 a 5 salas las cuales unirá con pasillos.
- En función del número de salas, se podrán generar un numero mayor o menor de objetos.

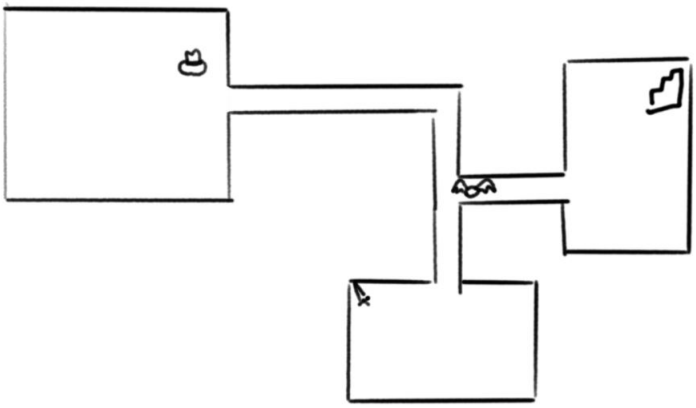


Figura 5.12.: Boceto que representa el diseño de la mazmorra.

- Los objetos y los enemigos solo se pueden generar en las salas, no en los pasillos. Aunque los enemigos, una vez generados, pueden moverse libremente por el escenario.
- La salida y la posición inicial del personaje también se generan en salas. El jugador cuando se crea conserva la cantidad de vida y puntos de magia con los que ha acabado el piso anterior, pero perderá todos los estados alterados que haya acumulado.
- Si el personaje había muerto anteriormente en el piso que se va a generar, se añade un objeto extra a la mazmorra con la cantidad de dinero que había dejado a la hora de morir.

Recorrido:

- Si caminas sobre las escaleras, aparece un mensaje dándote a elegir si continuar en el piso o acceder al siguiente.

5.2.5. Base

En la base se puede subir de nivel, gestionar el inventario, acceder a la tienda y regresar a la mazmorra. Es una zona segura sin enemigos, donde el jugador puede prepararse para adentrarse nuevamente en la mazmorra.

Baúl

- El baúl es un inventario presente en la base capaz de almacenar hasta 200 objetos. Es la principal forma de conservar objetos para futuras necesidades, pues estos no se pierden al morir.

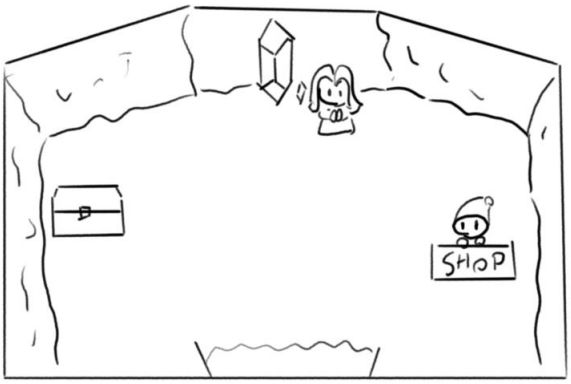


Figura 5.13.: Boceto que representa el diseño de la base.

- La forma de gestionar los objetos es muy similar a la que se tiene en el inventario, con la diferencia de que en vez de usar/equipar, la opción que da es la de coger o depositar.

Altar

- Misterioso altar en el que puedes ofrecer dinero para obtener poder, es decir, el dinero se usa para subir de nivel.

Mercader

- PNJ cuyo objetivo es ganar enormes riquezas. Gracias a él se pueden adquirir objetos a cambio de una suma de dinero. También le puedes vender los excedentes a un menor precio del de compra.

Salida

- Puerta por la que puedes regresar a la mazmorra. Una vez interactúas con esta, se muestra un menú con todos los puntos de control a los que se puede acceder.

5.2.6. GUI

El GUI representa la interfaz gráfica del jugador, donde se encuentran elementos como:

HUD

Muestra la información más importante del jugador durante la partida. Esta información es:

- Puntos de vida máximos y actuales.

- Puntos de magia máximos y actuales.
- Piso de la mazmorra en el que se encuentra el jugador

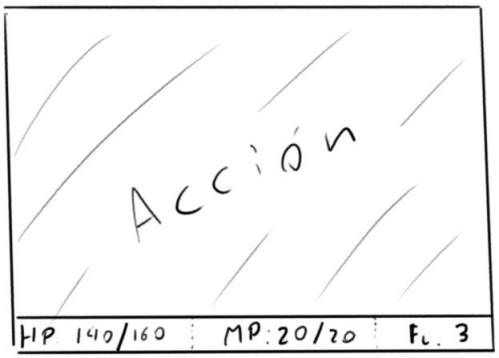


Figura 5.14.: Boceto que representa el diseño del HUD.

Menú de pausa

- Muestra una lista con todos los objetos que tienes en el inventario.
- Dentro del inventario se pueden equipar, usar, desechar o ver la información de cada objeto.

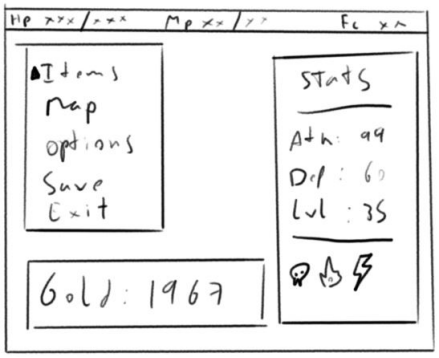


Figura 5.15.: Boceto que representa el diseño del menú de pausa.

Inventario

- Muestra una lista con todos los objetos que tienes en el inventario.
- Dentro del inventario se pueden equipar, usar, desechar o ver la información de cada objeto.

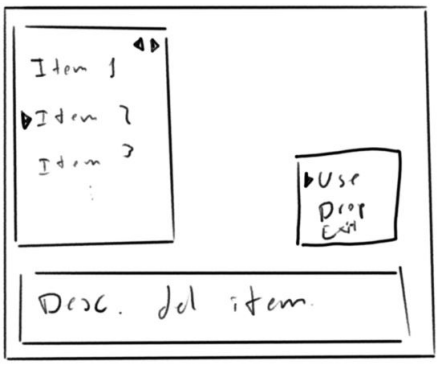


Figura 5.16.: Boceto que representa el diseño del inventario del jugador.

Mapa

- Muestra de forma simplificada las salas de la mazmorra que has visitado.

Salir

- Te devuelve a la acción.

5.3. Controles

D-Pad:

- Permite el movimiento en 8 direcciones por el mapa.
- Mueve el selector dentro de los menús.

A:

- En la mazmorra se utiliza para realizar la acción de atacar.
- En los menús sirve para realizar la acción de aceptar.
- En la base sirve para interactuar con los diferentes puntos de interés.

B:

- En la mazmorra (mantener) permite orientar al jugador sin moverlo
- En menús sirve para cancelar.

Start:

- Abre/Cierra el menú. En función de si se está en la mazmorra o en la base se mostrarán diferentes opciones.

Select:

- En la mazmorra usa el objeto/hechizo asignado.

5.4. Estilo visual

Project Rogue sucede en un entorno medieval con elementos mágicos, por lo que la mayoría de referencias visuales estarán centradas en este género. Iconografía como caballeros, magos, pociones y monstruos va a ser muy abundante durante la partida

Otro referente para el estilo de los gráficos es el videojuego The Legend of Zelda: Link's Awakening para Game Boy. Para ser más concretos, lo que interesa es el estilo visual de las mazmorras presentes en el juego, donde se mezclan elementos medievales y fantásticos.

5.5. Diégesis

5.5.1. Historia:

Introducción

En Project Rogue encarnas a un caballero errante en busca de la legendaria Torre Espejismo, situada en los bordes inexplorados del Gran Desierto. Allí espera obtener una cura para su hija, pues se dice que en la cima de la torre habita el Árbol de la Vida, cuyos frutos curan cualquier enfermedad. Lo que pocos saben es que en esa torre se canalizan poderes oscuros capaces de corromper el espacio tiempo y a los que habitan la propia torre.

Desarrollo

Al poco de adentrarse en la Torre Espejismo, el protagonista accede a una zona aislada del resto de la torre, la Base. En esta, conoce a una extraña mujer que le explica la situación de la torre. Al igual que nuestro protagonista, ha habido otros que han intentado escalar la torre, pero ninguno ha conseguido alcanzar la cima, en su lugar, han sucumbido a la torre y se han convertido en poderosas criaturas repletas de frustración que intentarán impedir que nadie avance por la torre. La extraña mujer también explica un extraño ritual con el que se puede incrementar el poder de un usuario a cambio de

un precio razonable. Dicho ritual también garantiza que, ante las puertas de la muerte, el usuario que ha realizado el pacto volverá a la base sano y salvo, con la condición de pagar un precio.

En la Base también conoce a un mercader cuya motivación parece ser la de encontrar objetos únicos y regresar de la para venderlos a un buen precio. Este le comentará al protagonista de la existencia de un extraño baúl capaz de almacenar una gran cantidad de objetos cuyo contenido solo puede ser recogido por la persona que dejó el objeto en primer lugar.

Conforme avanza la aventura, el protagonista se va enfrentado a enemigos cada vez más poderosos, regresando a la Base de forma regular para abastecerse y aumentar sus características.

Conclusión

Tras alcanzar la cima, el protagonista ve a la extraña mujer junto al Árbol de la Vida que le explica al protagonista que el fruto de ese árbol a parte de curar todos los males, es un sello mágico que mantiene encerrado un antiguo mal que es el responsable del estado actual de la torre. Sin dudar un momento el protagonista se decide a coger la fruta y, tras cogerla, empieza la batalla final. Después de salir victorioso, el protagonista queda gravemente herido, pero gracias a su perseverancia, logra entregarle la Fruta de la Vida a su hija.

5.5.2. Protagonista/s:

Caballero errante (Protagonista)

El nombre no es importante (se lo da el jugador). Con una única idea en mente, este caballero se enfrentaría a todos desafíos que hagan falta y lo sacrificaría todo para lograr su objetivo.



Figura 5.17.: Boceto que representa el diseño del personaje principal.

5.5.3. PNJs hostiles:

Murciélago Gigante

Este murciélago de gran envergadura es muy agresivo, pero no supone una gran amenaza cuando va en solitario. Sin embargo si va en grupo hay que extremar las precauciones, pues tienden a atacar desde todos los ángulos posibles.



Figura 5.18.: Boceto que representa el diseño del Murciélago Gigante.

Ojo Diabólico

Este ojo de gran tamaño se dedica a recorrer cada piso de la mazmorra en busca de un intruso y es extremadamente agresivo en caso de encontrarlo. No supone una gran amenaza con el equipamiento adecuado.



Figura 5.19.: Boceto que representa el diseño del Ojo Diabólico.

Mago Oscuro

Este forma parte de una antigua tribu de magos que decidió adentrarse en la torre con el propósito de mejorar sus artes oscuras. Con el tiempo ha acabado por corromperse y ahora se dedica a proteger la torre. Su defensa no es muy elevada, pero es capaz de realizar poderosos ataques mágicos.

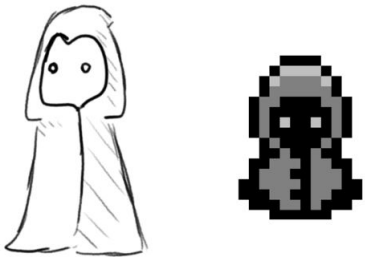


Figura 5.20.: Boceto que representa el diseño del Mago Oscuro.

Elemental de Roca

Las poderosas energías oscuras presentes en la torre han sido capaces de afectar incluso a la materia inorgánica del lugar, generando unos poderosos autómatas de roca que atacarán a cualquier intruso. Estos elementales tienen una gran defensa física, por lo que se recomienda usar ataques mágicos con ellos.

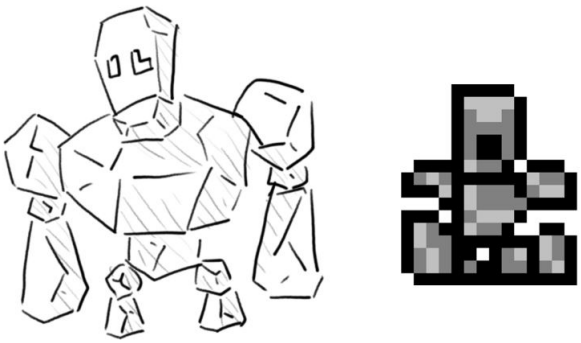


Figura 5.21.: Boceto que representa el diseño del Elemental de Roca.

5.5.4. PNJs amistosos:

Mujer extraña

Esta extraña mujer es todo un enigma, lo único que se sabe de ella es que se encarga de realizar el ritual del cristal. Además tiene extensos conocimientos sobre la torre y lo que habita en esta.



Figura 5.22.: Boceto que representa el diseño de la Mujer Misteriosa.

Rudolf el Mercader

Rudolf es un mercader amistoso que fue a la torre para obtener tesoros y artefactos únicos de gran valor. Actualmente es el único mercader de la torre lo que le permite acaparar todo el mercado de la misma, vendiendo y comprando los objeto y el equipo de los aventureros que pasan por allí.

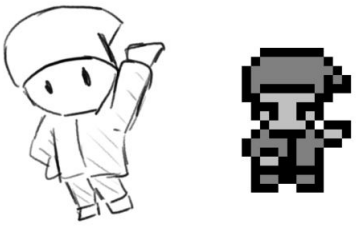


Figura 5.23.: Boceto que representa el diseño del Mercader.

5.6. Herramientas de Desarrollo

Para realizar un juego de Game Boy no son necesarias demasiadas herramientas de desarrollo, de hecho, con un solo bloc de notas se podría generar un ejecutable. Esto, pese a ser posible no es recomendable, debido a que existen múltiples herramientas que permites agilizar el proceso de desarrollo. Por eso, en este apartado se van a analizar todas las herramientas usadas en el desarrollo de este proyecto.

5.6.1. Ensamblador

Para generar un programa ejecutable por la Game Boy, este debe estar escrito en código máquina. Un archivo escrito de esta forma no es más que un archivo binario donde cada byte o conjunto de bytes representan una instrucción admitida por el procesador de la Game Boy.

Con el fin de facilitar la programación a bajo nivel, se hace uso del lenguaje ensamblador. Este tipo de lenguaje adapta la forma de representar las instrucciones del código máquina por otras que incorporan texto para representar cada instrucción. Pese a que incorpore texto, el lenguaje ensamblador sigue siendo bastante arcaico, por lo que se recomienda usar comentarios indicando cada acción realizada paso a paso, sobre todo si se trabaja en grupo.

```
115 _generate_random_min_max:
116
117     ld d, a                     ;D -> mascara binaria
118     push bc
119 .generate_num:
120
121     push de
122     call _generate_random
123     pop de
124     and d                       ;aplicamos la mascara
125
126     pop bc                      ;BC -> Min, Max value
127     cp b                        ;Asegura que el minimo es el establecido por B
128     jr c, .generate_num
129     push bc
130
131     push af                     ;Incrementamos el máximo en 1 para poder hacer la comparacion bien
132     ld a, c
133     inc a
134     ld c, a
135     pop af                      ;C = C + 1
136
137     cp c                        ;Asegura que el maximo es C - 1
138     jr nc, .generate_num
139
140     pop bc
141
142     ret
143
```

Figura 5.24.: Extracto del código ensamblador del proyecto.

Pese a ser una mejora considerable respecto al código máquina, el lenguaje ensamblador no puede generar un archivo ejecutable por si mismo, necesita de una herramienta llamada ensamblador. No hay que confundir el código ensamblador con el ensamblador pues no son lo mismo, el primero de estos simplemente es un lenguaje de programación, mientras que el segundo es un programa que traduce el código ensamblador a código máquina capaz de ser ejecutado por la consola.

En el caso de este proyecto se ha usado el toolkit de RGBDS para el proceso de ensam-

blaje. Este toolkit incluye un ensamblador llamado RGBASM, un enlazador RGBLINK y un corrector RGBFIX.

RGBDS no es la única herramienta para ensamblar el juego, existen otras como ISAS o VASM, siendo el primero de estos el ensamblador oficial usado por Nintendo y el segundo un ensamblador genérico para procesadores como el Zilog Z80 que también es capaz de ensamblar para la Game Boy.

5.6.2. Editor de texto

Para escribir y editar código se ha usado el editor de texto Visual Studio Code. La elección de este editor viene dada por su anterior uso en otros proyectos, su capacidad para sincronizarse con un repositorio de GIT y la cantidad de extensiones que existen para el mismo.

Como configuración adicional se ha instalado la extensión RGBDS Z80. Esta una de las diversas extensiones disponibles que permiten visualizar el lenguaje ensamblador de procesadores como el Zilog Z80 con un código de colores que distingue las diferentes instrucciones.

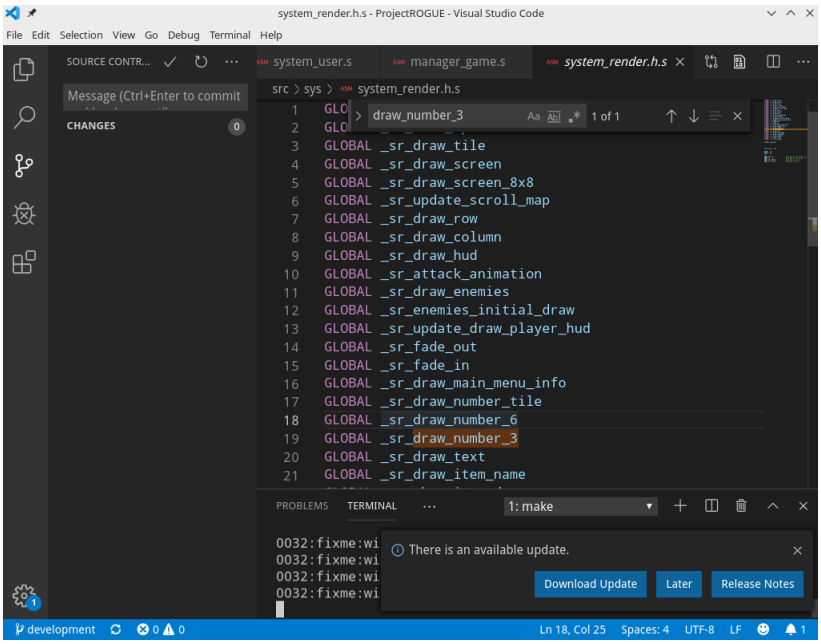


Figura 5.25.: Captura de pantalla del Visual Studio Code.

5.6.3. Editor de imágenes

Como editor de imágenes para realizar los diferentes dibujos y esquemas se ha usado Krita. Este es un programa gratuito de código libre que sirve principalmente para realizar dibujos, arte conceptual e ilustraciones entre otras. Pese a que este es el principal objetivo de la herramienta, también se puede usar como sustituto a programas como Photoshop o Gimp.

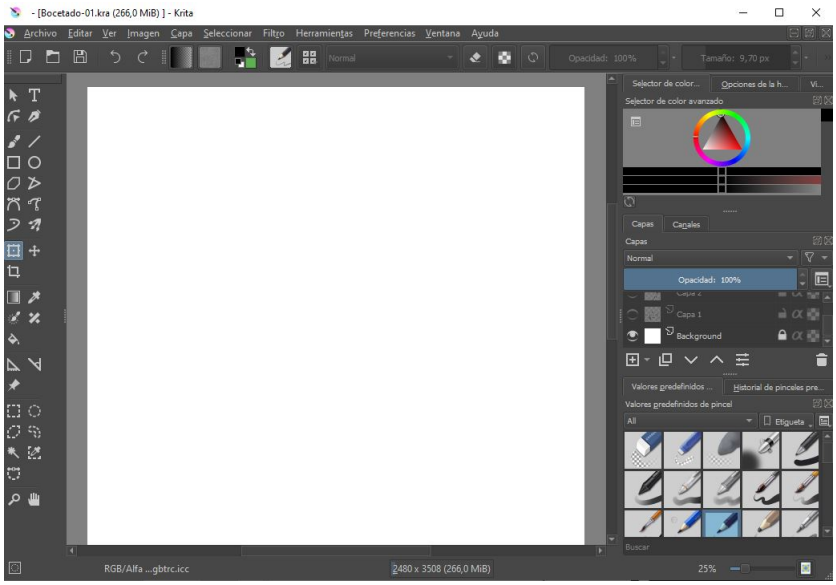


Figura 5.26.: Captura de pantalla de Krita.

5.6.4. Editor de tiles y sprites

Para diseñar los tiles y sprites presentes en el juego se ha utilizado la herramienta Game Boy Tile Designer. Esta herramienta está pensada para diseñar tiles con las restricciones de la Game Boy lo que permite visualizar cómo se van a representar los mismos a la hora de introducirlos al juego. Cuenta con un modo de visualización de hasta 4 tamaños, siendo estos de 8x8, 8x16, 16x16 y 32x32 píxeles respectivamente. También cuenta con una paleta de colores configurable para un mayor control a la hora de diseñar los distintos elementos.

Por último y la mayor ventaja de esta herramienta es que puede exportar los tiles generados directamente a un fichero binario que puede ser incluido directamente en el juego sin necesidad de realizar nungún cambio.

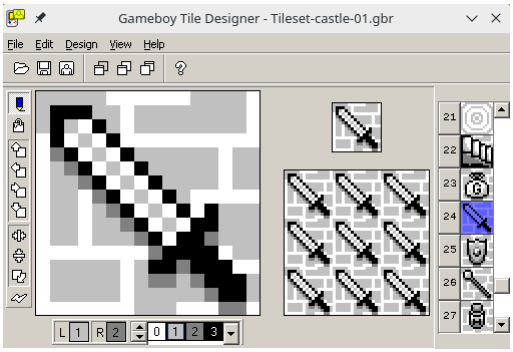


Figura 5.27.: Captura de pantalla del Game Boy Tile Designer.

5.6.5. Editor de mapas

Pese a que el juego cuenta con mapas generados procedimentalmente, durante las primeras fases del desarrollo fue necesaria una herramienta que permitiera generar y editar dichos mapas. La herramienta que se ha seleccionado para ese cometido es Game Boy Map Builder.

Esta herramienta permite generar mapas de un tamaño de hasta 1024 x 1024 tiles. Para dibujar los tiles del mapa es necesario un tileset, que se puede obtener directamente de la herramienta Game Boy Tile Designer. Esta relación entre los programas permite que el diseño de los mapas sea mucho más sencilla y directa pues cada cambio en el tileset se ve reflejado automáticamente en el propio mapa.

Al igual que con la herramienta anterior, Game Boy Map Builder también puede exportar los mapas creados en un formato que después pueda ser incorporado en el juego de forma directa. Dicho formato se puede configurar de muchas formas diferentes, siendo capaz de adaptarse a lo que el desarrollador desee.

Game Boy Map Builder también se ha usado a la hora de generar el fondo para los menús y el fondo de la pantalla de inicio.

5.6.6. Herramientas de testeo y depuración

Por último, para probar y depurar el juego se han usado dos herramientas principales: un emulador y un flashcart.

Emulador

El emulador es una herramienta muy práctica a la hora de desarrollar un juego para la Game Boy, pues permite ejecutar el juego directamente desde el ordenador sin necesidad de un hardware externo. También es recomendable probar el juego en el sistema original, en este caso la Game Boy, o en otros emuladores disponibles. Esto es debido a que un

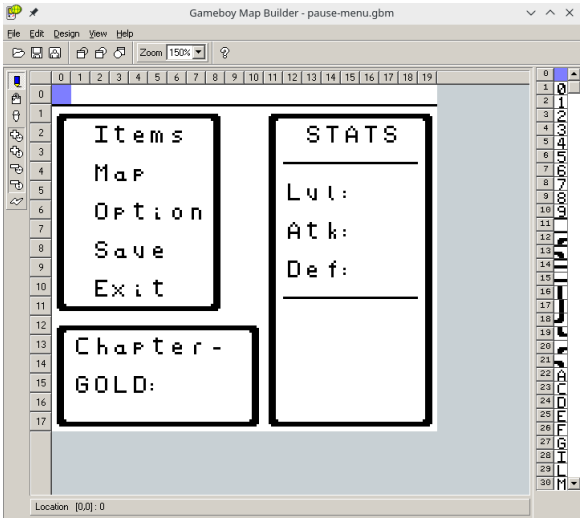


Figura 5.28.: Captura de pantalla del Game Boy Map Builder.

emulador no puede simular a la perfección todas las características de la videoconsola y se pueden provocar o evitar errores que sí están presentes en el hardware original.

En el caso de este proyecto, el emulador que se ha usado es BGB. Este emulador está considerado como uno de los que mejores, pues simula el sistema de la Game Boy de forma prácticamente perfecta y incluye una serie de herramientas de depuración muy completas que se verán a continuación.

La principal herramienta de depuración que ofrece este emulador es el debugger. En este se muestran cuatro grupos principales de información:

- El desensamblador. Está situado en la esquina superior izquierda y, de forma inversa a lo que realiza el ensamblador, el desensamblador parte del binario del juego y a partir de este, genera el código ensamblador que se puede observar. Esta sección es la que se usa principalmente para depurar el juego debido a que tienen la capacidad de detener la acción en un punto determinado usando breakpoints. Tumben se puede avanzar la ejecución instrucción a instrucción pulsando la tecla F7 para comprobar el valor de los registros y la memoria, detectando posibles errores en el proceso.

- Visualizador de memoria. En la zona situada debajo del desensamblador se puede observar el estado de toda la memoria del juego. Este visualizador de memoria se suele usar junto al desensamblador para depurar los datos del juego y como se modifican a lo largo de la ejecución.

- Registros y datos del procesador. La zona situada en la esquina superior derecha

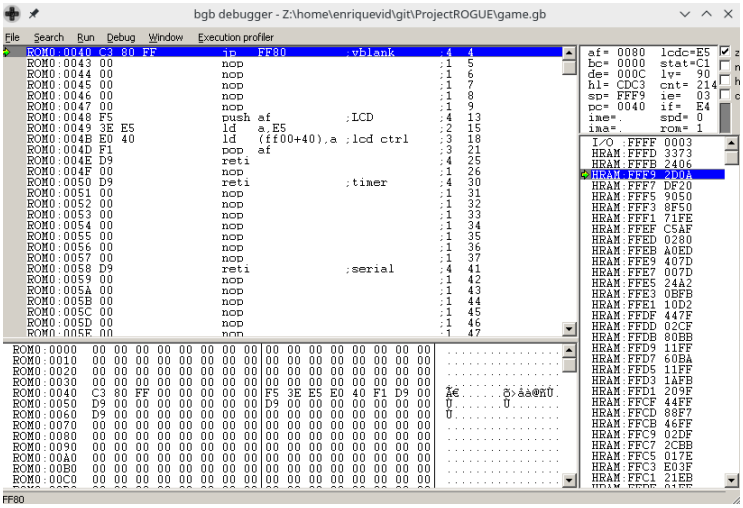


Figura 5.29.: Captura de pantalla del debugger de BGB.

está dedicada a mostrar el valor de lo registros así como el valor de las flags. Al usarse junto al desensamblador, la información que aportan los registros para depurar es muy relevante pues permite comprobar si se ha cargado bien la información indicada en el código.

- La pila. Este es el último de los elementos presentes en el debugger del emulador y su función es la de mostrar todos los registros almacenados en la ella de manera ordenada.

A parte de la función de debugger, BGB también añade una ventana donde se puede comprobar el estado de cada una de las partes de la VRAM de directa.

La primera pestaña de esta ventana es el BG map. En esta se pueden ver de forma gráfica los tiles que componen el fondo y un rectángulo que representa la parte del mapa que se está visualizando. También da información sobre el tile situado bajo el cursor.

La segunda pestaña muestra todos los tiles cargados en memoria. En el caso de que el tile haya sido cargado en el BG map, este se muestra con la paleta que le correspondería, mientras que los que no se han cargado se ven en escala de grises. También muestra información relativa al tile en cuestión.

En la tercera pestaña se muestra una tabla con todos los sprites y sus atributos. De forma similar que pasa con los tiles, cuando un sprite está por fuera de la zona visible de la pantalla, este se muestra tachado.

Por último, la pestaña que queda muestra una lista con todas las paletas de colores del juego y sus valores. En función de si se está en modo de Game Boy o Game Boy color, se muestra un número paletas activas diferente.

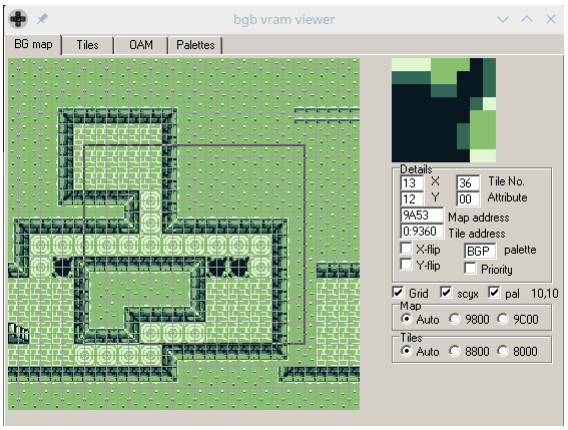


Figura 5.30.: Captura de pantalla del apartado BG map del emulador BGB.

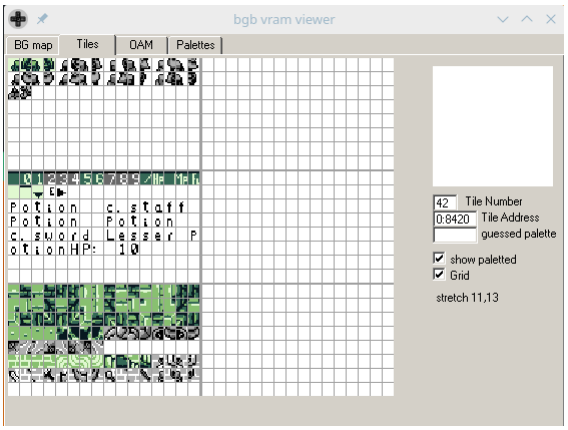


Figura 5.31.: Captura de pantalla del apartado Tiles del emulador BGB.

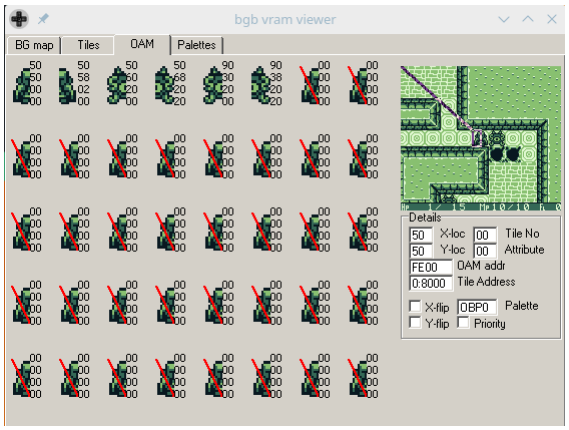


Figura 5.32.: Captura de pantalla del apartado OAM del emulador BGB.

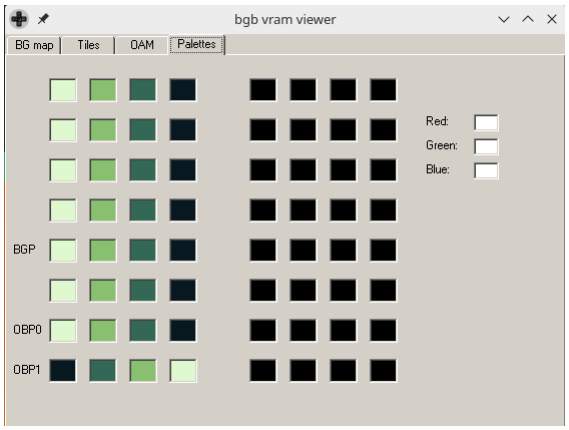


Figura 5.33.: Captura de pantalla del apartado Pelettes del emulador BGB.

Flashcart

Otro elemento usado para probar el correcto funcionamiento del juego es el uso de un flashcart de Game Boy. Este cartucho permiten ejecutar copias de juegos de Game Boy y Game Boy Color directamente en un sistema compatible, teniendo la certeza de que el juego va a funcionar como realmente debe. Para hacer uso del flashcart es necesario almacenar una copia del juego en la una tarjeta micro SD que posteriormente se conectará al propio cartucho.

En el caso de este proyecto, se ha usado el flashcart EZ-FLASH junior junto a una consola Game Boy Advance. Pese a ser una versión superior, la Game Boy Advance y su revisión Game Boy Advance SP son totalmente compatibles con los juegos de Game Boy y Game Boy Color a nivel de harware por lo que no debería haber diferencia.



Figura 5.34.: Fotografía de la flashcart EZ-Flash jr.

6. Desarrollo

A continuación se va a describir cada parte del desarrollo del proyecto dividido en las iteraciones realizadas, que duran aproximadamente dos semanas de desarrollo, aunque las hay que duran más.

6.1. Iteración 1: Elección de Ensamblador

Una de las elecciones que se deben hacer al comienzo de un proyecto como este, es elegir qué ensamblador se va a utilizar debido a que es una de las herramientas más importantes de todo el proyecto.

Durante el periodo de aprendizaje se han probado un total de dos ensambladores para la realización del juego. El primero de estos es VASM, que fue el recomendado por la primera fuente de información disponible relacionada con desarrollo de Game Boy [3] debido a su capacidad para ensamblar para varios procesadores. De entre todos los procesadores para los que puede ensamblar, el Zilog Z80 es el que se debe elegir pues mediante una instrucción a la hora de compilar, lo puede hacer también para el Sharp LR35902 empleado en la Game Boy y Game Boy Color.

El segundo de los ensambladores que se han probado es RGBASM debido a su elevado uso en la comunidad de desarrolladores de Game Boy. Este forma parte del toolkit RGBDS, un conjunto de herramientas que tiene como principal objetivo el desarrollo de software para la familia Game Boy, y se compone por tres aplicaciones individuales: RGBASM, el ensamblador; RGBLINK, el enlazador (linker) y RGBFIX, se encarga de corregir la cabecera del juego.

Tras un tiempo probando ambos ensambladores y pese a tener prácticamente las mismas funcionalidades, se decidió usar RGBDS como ensamblador del proyecto. Esto fue debido a su mayor y mejor expresada documentación [4], y a características como la capacidad de declarar secciones de código en posiciones generales como la ROM0 y que sea el proceso de enlazado el que le de una posición final (esta declaración también se puede realizar para los diferentes bancos de memoria de la consola). También, aunque en menor medida el uso RGBFIX fue un elemento decisorio, pues permite corregir de una forma sencilla las cabeceras para generar un juego funcional en una consola original. Pese a todo, de elegir VASM como ensamblador, también se puede emplear RGBFIX.

Una vez elegido RGBDS como ensamblador se va a explicar de forma resumida el proceso a seguir para generar un archivo capaz de ejecutarse en una consola Game Boy o emulador. Dicho proceso se puede observar de forma simplificada en la figura [6.1]

El primer paso a la hora de generar un archivo ejecutable usando RGBDS es RGBASM, el ensamblador. Este se encarga de generar un archivo binario (con terminación '.o') por



Figura 6.1.: Diagrama de la linea de ensamblado

cada archivo escrito en lenguaje ensamblador (terminados en '.s') que se encuentre en la ruta especificada. La terminación de cada archivo es indiferente, pero en el caso de este proyecto se van a usar las anteriormente mencionadas para referirse a cada tipo de archivo. A continuación se muestra un código de ejemplo [Listado 6.1]:

Listado 6.1: RGBASM Código de Ejemplo

```
1 rgbasm -o obj/main.o src/main.s
```

El segundo paso es unir todos los binarios en un solo archivo con terminación '.gb' o '.gbc' ya sea para Game Boy o para Game Boy Color. Esta acción se hace mediante el linker RGBLINK. En este paso también se puede crear un archivo muy importante a la hora de depurar mediante la instrucción '-m', pues en este se muestran las direcciones de memoria de todas las etiquetas declaradas en el código así como el tamaño de cada una de las secciones declaradas. A continuación se muestra un código de ejemplo [Listado 6.2]:

Listado 6.2: RGBLINK Código de Ejemplo

```
1 rgblink -m game.map -o game.gb game.obj
```

El tercer y último paso del proceso es ejecutar RGBFIX con el archivo generado por el RGBLINK. Este paso es necesario para generar los 'checksums' necesarios para que el juego funcione en una consola real, aunque también permite modificar ciertos elementos como el tamaño del cartucho, la región del juego, el gameID y en general todos los parámetros de la cabecera del juego. A continuación se muestra un código de ejemplo [Listado 6.3]:

Listado 6.3: RGBFIX Código de Ejemplo

```
1 rgbfix -v -p 0 game.gb
```

En caso de querer más información sobre las instrucciones de cada uno de los pasos realizados, se pueden ver en los anexos de RGBASM[A.1], RGBLINK[A.2] y RGBFIX[A.3].

6.2. Iteración 2: Makefile y Código Inicial

Para la realización de este proyecto se ha decidido estructurar el código en diferentes archivos que agrupan diferentes clases o funcionalidades (con terminación .s). También se ha decidido crear un archivo de cabecera (con terminación.h.s) por cada archivo de código, donde se declaran todos los elementos globales (variables, constantes y funciones) para que los demás archivos puedan acceder a los mismos. Los archivos de cabecera no son necesarios dado que las funciones que cumplen se pueden hacer directamente en los archivos de código, pero aportan un mayor nivel de desacoplo, lo que permite una mayor claridad a la hora de programar.

6.2.1. Makefile

Debido a la estructura de archivos, la forma más productiva de generar un ejecutable es mediante la creación de un archivo makefile que se encargue de ensamblar todos los archivos de código así como de enlazar los binarios resultantes para generar un archivo ejecutable por la Game Boy, y de corregir la cabecera del ejecutable.

El primer paso que realiza el makefile es buscar todos los archivos de código con terminación '.s', descartando sus cabeceras (.h.s) mediante la función 'find' del shell de Linux. Una vez localizados todos los archivos, los ensambla uno a uno empleando RGBASM, generando sus respectivos binarios con terminación '.o'. Acto seguido, de la misma manera que ha buscado todos los archivos '.s', busca y enlaza (linker) usando RGBLINK todos los binarios, generando los ejecutables para Game Boy y Game Boy Color ('.gb' y '.gbc'). Por último se aplica el RGBFIX a los dos ficheros para dejarlos listos para jugar. En la figura [6.2] se puede observar un diagrama con el proceso que realiza el makefile simplificado.

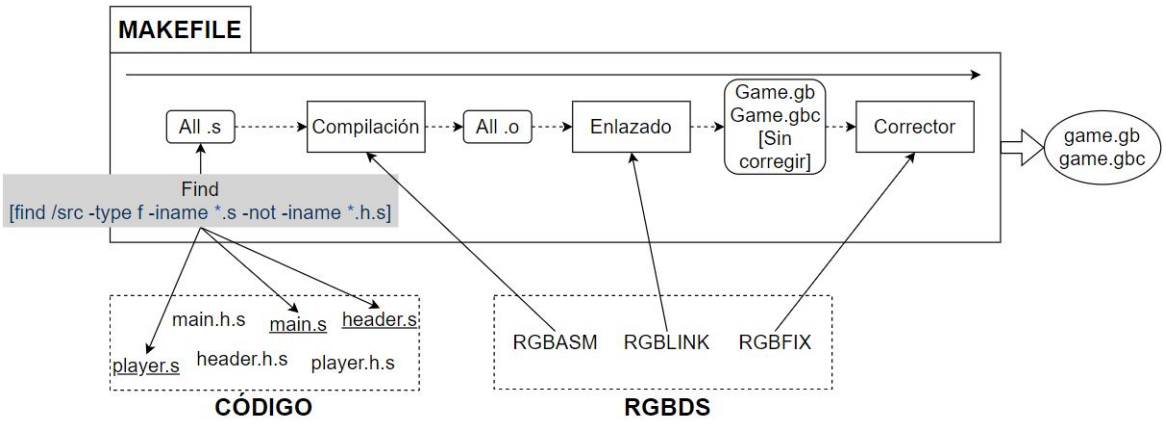


Figura 6.2.: Diagrama del funcionamiento de un makefile

6.2.2. Código Inicial

Ahora que el makefile está terminado, es momento de generar el código necesario para que el juego se pueda ejecutar correctamente, dejando el proyecto preparado para programar las diferentes mecánicas. Como ya se ha mencionado antes, se ha decidido realizar una separación de archivos por funcionalidades y, debido a esto, la cabecera del juego tiene su propio archivo (header.s) que será incluido al archivo principal durante el proceso de enlazado. Tal y como se mencionó en el apartado de memoria ROM del Marco Teórico [2.2.8], cada dirección de memoria dentro de la cabecera tiene una función asignada, por lo que si hay algún error en esta parte del código, el juego puede no llegar a ejecutarse correctamente.

Estos errores de la cabecera, se pueden corregir de manera externa empleando RGBFIX, sin embargo, hay partes que no pueden ser corregidas por este. Estas partes son las que abarcan de la dirección 0040_{hex} a la 0103_{hex}, pues tratan las interrupciones así como el punto de entrada al código del juego.

Si las activamos, las interrupciones ejecutaran la porción de código asociado a su dirección de memoria cada vez que se cumpla con una condición específica. Por ejemplo, la interrupción del Vblank se activa cada vez que el dibujado de la pantalla entra en la porción del Vblank. Dejando de lado las interrupciones, la otra dirección de memoria a tener en cuenta es el punto de entrada pues va a indicar en qué dirección de memoria empieza el código del juego.

Una vez creada la cabecera comienza el fichero principal, denominado como 'main'. Al comienzo de este se realizan las configuraciones iniciales, tales como posicionar el puntero de la pila (SP),y configurar la pantalla, aunque más adelante se añadirán otras configuraciones iniciales a medida que vayan siendo necesarias.

El puntero a la pila suele ponerse en la dirección de memoria $FFFF_{\text{hex}}$ pues es donde acaba la HRAM o High RAM. El hecho de situar el puntero es esta dirección de memoria es debido al funcionamiento que tiene, ya que cada vez que se guarda algo en la pila, el puntero accede a la dirección anterior en vez de a la posterior, es decir, que pasa de la $FFFF_{\text{hex}}$ a la $FFFE_{\text{hex}}$.

La configuración de la pantalla se centra en la dirección de memoria $FF40_{\text{hex}}$ ya que se trata del control del LCD. Cada bit del byte que contiene esta dirección representa una acción distinta si están a 1 o a 0, por ejemplo el bit 7 (primero empezando por la izquierda) enciende y apaga la pantalla dependiendo si está a 1 o a 0. Hay otras funciones asociadas al control del LCD tal y como se puede ver en la tabla [6.1], pero estas serán explicadas conforme vayan apareciendo en el proyecto.

El último paso para tener una base ejecutable es realizar un bucle infinito, esto se hace para que el juego no ejecute partes de código no deseadas mientras se ejecuta. Este bucle, será denominado game_loop y aunque ahora mismo no tiene nada, acabará por tener todo el código relacionado con el apartado jugable.

Tabla 6.1.: Funciones del control LCD [0xFF40]

Bits	**Acciones 0/1**
Bit 7	Control del LCD apagado/encendido
Bit 6	Window Tile Map Display Select [0x9800 - 0x9BFF]/[0x9C00 - 0x9FFF]
Bit 5	Window Display off/on
Bit 4	BG and Window Tile Data Select [0x8800 - 0x97FF]/[0x8000 - 0x8FFF]
Bit 3	BG TIle Map Display Select [0x9800 - 0x9BFF]/[0x9C00 - 0x9FFF]
Bit 2	OBJ (Sprite) Size 8x8px/8x16px
Bit 1	OBJ (Sprite) Display off/on
Bit 0	BG and Window Display off/on

6.3. Iteración 3: Dibujado por Pantalla

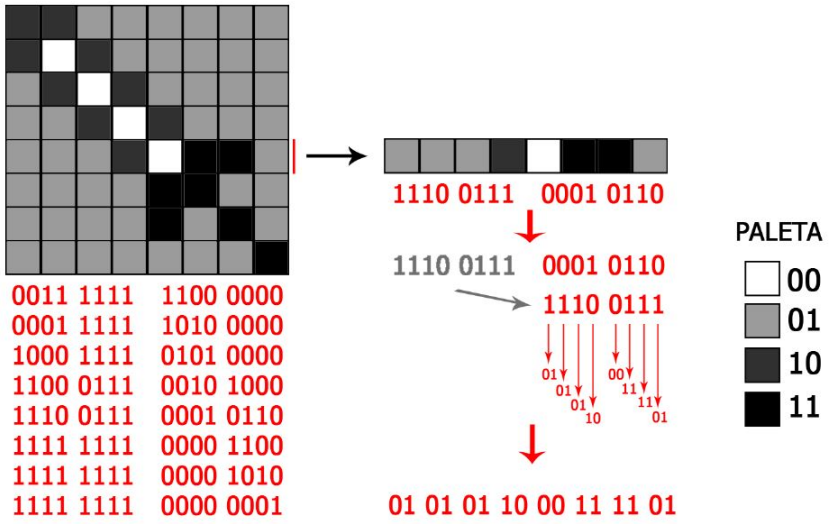
Hasta ahora se tiene una base que se puede ejecutar en una Game Boy o en un emulador pero no realiza ninguna acción, así que el siguiente paso es mostrar imágenes por pantalla en forma de tiles y sprites. A todos estos procedimientos relacionados con el dibujado se les ha asignado el archivo render.s.

El primer paso a la hora de dibujar es declarar las paletas de colores que van a ser usadas. En el caso de la Game Boy las paletas se declaran en las direcciones de memoria $FF47_{hex}$ para los tiles, y $FF48_{hex}$ y $FF49_{hex}$ para los sprites. En cuanto a la codificación, en cada dirección de memoria hay que escribir un byte que define los colores de la paleta. Este byte, se puede dividir en cuatro parejas de dos bits cada una de estas parejas puede tomar un valor del 0 al 3 en binario siendo el 0 blanco, el 3 negro y el resto grises intermedios. Las posición en la que se pongan los colores de la paleta es relevante pues tal y como se explicara mas adelante, los tiles y los sprites se codifican en función de las posiciones de la paleta, no en función de los colores de la misma.

Antes de poder dibujar nada hay que dejar claros varios conceptos, siendo el primero de ellos cómo se codifica un tile para que pueda ser usado más adelante, usando la figura [6.3] como base para le explicación. Para empezar, cada tile está formado por 16 bytes, 2 por cada fila de 8 píxeles. Si se pasan a binario los dos bytes de una sola fila se pueden observar dos grupos de 8 bits cada uno, y si estos grupos se ordenan uniendo el primer bit del primer grupo con el primer bit del segundo grupo, tal y como se muestra en la figura, se forman parejas de bits que forman los números del 0 al 3 en binario (este proceso se realiza para todos los bits de cada grupo). Estos números hacen referencia color de la paleta que van a usar, por ejemplo el 00 usará el primer color que se haya declarado en la paleta (bits 0 y 1 del byte de la paleta), mientras que el 11 usará el último color de la paleta (bits 6 y 7 del byte de la paleta) independientemente de los colores que hayan en las paletas.

Una vez tenemos los tiles, estos se almacenan inicialmente en la ROM, y para poder

Figura 6.3.: Esquema de la codificación de un tile



ser usados, deben copiarse a la sección de la VRAM o Vídeo RAM que engloba de la dirección 8000_{hex} a la $97FF_{\text{hex}}$. Este espacio de direcciones se divide a su vez en tres secciones, de las cuales se hablará más adelante. Ahora bien, para poder copiar los tiles a la VRAM se debe hacer en un periodo de tiempo específico durante el que la consola deja de leer de VRAM y permite escribir datos en la misma, estos periodos de tiempo son conocidos como el Hblank y el Vblank. El Hblank es el periodo de tiempo que hay entre que la consola termina de dibujar una linea de la pantalla y comienza la siguiente linea, mientras que el Vblank es es periodo que tarda entre que dibuja la última linea y vuelve a comenzar por la primera. Teniendo esto en cuenta, en la Game Boy existe la dirección de memoria $FF41_{\text{hex}}$ conocida como LCD Stat. A la hora de leer de esta dirección, los bits 0 y 1 nos indican el estado del dibujado de la pantalla. Por tanto para copiar un tile de ROM a VRAM hay que comprobar si se está en periodo de acceso, en caso de estarlo, se procede a copiar los bytes que conforman el tile, y en caso de no estar disponible la escritura en VRAM, lo que se hace es esperar y volver a comprobar el estado.

Otra opción a la hora de cargar los tiles a VRAM es la de apagar la pantalla, debido a que cuando la pantalla está apagada se tiene un acceso total para escribir en VRAM, por lo que se puede omitir el paso de la comprobación del Vblank y el Hblank. Esto se consigue modificando el bit 7 de la dirección $FF40_{\text{hex}}$, que como se ha visto anteriormente se encarga el control del LCD, la única condición antes de apagar la pantalla es asegurarse de que la misma este en el periodo de Vblank, pues si no lo está, se puede dañar la misma de forma física.

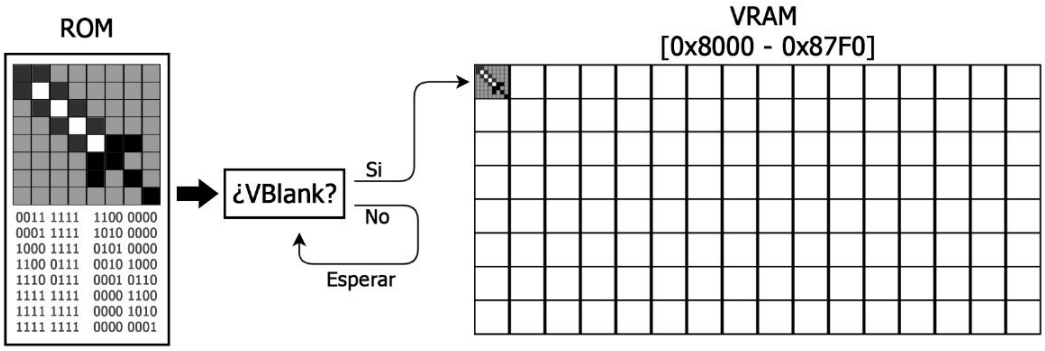


Figura 6.4.: Esquema de cargar un tile en VRAM

Ahora, con los tiles cargados en la VRAM es hora de dibujarlos por pantalla, para lo que se deben entender una serie de conceptos. El primero de estos es que cada tile dentro de la VRAM tiene un identificador asociado que se relaciona con la posición que ocupa, es decir que el primer tile de la VRAM tiene el identificador 00_{hex} mientras que el doceavo tiene el identificador $0B_{\text{hex}}$. Esto es importante pues a la hora de dibujar los tiles, se hará uso de ese identificador para elegir que tile se dibuja.

El siguiente concepto es el del BG map, espacio que usa la Game Boy para establecer los tiles que se van a dibujar por pantalla. Este es un espacio de 32x32 bytes que engloba de la dirección 9800_{hex} a la $9BFF_{\text{hex}}$ o alternativamente de la dirección $9C00_{\text{hex}}$ a la $9FFF_{\text{hex}}$, donde cada byte hace referencia al identificador de los tiles que se almacenan en la VRAM, lo que permite tener un mapa de 32x32 tiles cargados simultáneamente. La dirección que ocupa el BG map, tal como se ha mencionado antes, puede variar modificando el bit 3 de la dirección del control LCD ($FF40_{\text{hex}}$). Esto se hace para combinar el BG map con la denominada Window, que se explicará más en detalle en la iteración 9 [6.9].

El último concepto a tener en cuenta antes de dibujar es el de la View. Esta vista, delimita lo que se puede ver del BG map en cada momento y es del mismo tamaño de la pantalla de la Game Boy (160 x 144 px). Esta vista tiene la capacidad de moverse por el BG map, mostrando los tiles del mismo por la pantalla con la cualidad de no estar regida a la cuadricula a la que sí está regida el BG map, lo que permite hacer movimientos fluidos a la hora moverse por el mismo. Por poner un ejemplo, la View se puede considerar como una cámara que se mueve por un fondo. Otra cualidad de la View es que cuando llega a un extremo del BG map, vuelve a comenzar por el extremo contrario, es decir si la View se pasa por la derecha del BG map, la parte que se pasa muestra la parte izquierda del mismo.

Toda la información referente a la memoria de vídeo se puede encontrar en el apartado de memoria de vídeo del Marco Teórico [2.2.9].

Con todo esto claro, para cargar un tile de VRAM al BG map se realiza un proceso similar al de cargar un tile de ROM a VRAM siendo el primer paso esperar al Vblank o Hblank. La diferencia radica en que a la hora de pasar el tile de VRAM al BG map lo que se pasa no es el contenido del tile, sino el identificador del mismo.

Ahora bien, como ya se ha mencionado, la Game Boy trabaja con tiles de 8x8 px, pero en este proyecto la idea es trabajar con tiles de 16x16 px para realizar unos gráficos más cuidados. Para solucionar este problema se debe crear un procedimiento llamado draw_tile capaz de juntar cuatro tiles de 8x8px y formar uno de 16x16 px tal y como se muestra en la figura [6.5]

Este procedimiento funciona recibiendo un identificador del tile y un puntero a la posición del BG map donde se quiere empezar a dibujar. Una vez tiene todos los datos dibuja el primer tile de 8x8 px, incrementa en 1 el puntero y lo dibuja nuevamente. Ahora, para dibujar el tercer tile, incrementa el puntero en el ancho del mapa - 1, y por último vuelve a incrementar en uno el putero para dibujar el ultimo de los tiles.

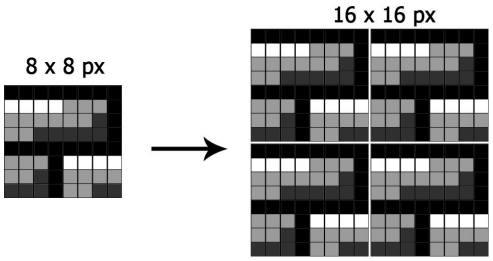


Figura 6.5.: Conversión de 8x8px a 16x16px

El siguiente paso tras el dibujado de tiles es el dibujado de sprites. Estos al contrario que los tiles, se usan para los elementos dinámicos del juego, debido a que no están regidos por ninguna cuadricula y a que se muestran por encima de los tiles. Otra característica de los sprites es que su posición es relativa a la View, y no al BG map teniendo en cuenta unos márgenes en los lados superior e izquierdo de 8 px, lo que provoca que si se sitúa el sprite en la posición (0,0), no se muestre mientras que si se sitúa en la (8,8) se muestre en la esquina superior izquierda. Los sprites también tienen limitaciones y es que si hay más de 10 sprites en una misma linea de dibujado, estos empezarán a parpadear debido a limitaciones de la consola.

Teniendo estas características en cuenta, el primer paso a la hora de dibujar sprites es habilitarlos, para lo que se hará uso de los bits 1 y 2 del control LCD ($FF40_{hex}$), donde el bit 1 permite activar o desactivar los sprites y el bit 2 permite seleccionar sprites de 8x8px o de 8x16px (no puede haber sprites de 8x8 y de 8x16 a la vez). En el caso de este juego se hará uso de sprites de 8x16px pues permiten crear sprites de 16x16px de forma mas sencilla.

En cuanto a la codificación, los tiles y los sprites se codifican de la misma forma, de

hecho, los sprites también utilizan los tiles copiados a la VRAM así que tanto tiles como sprites sacan la imagen que muestran de las mismas direcciones. La diferencia radica a la hora de dibujarlos, pues hacen uso de la sección de la memoria denominada como OAM (Object Attribute Memory) que se localiza de la dirección $FE00_{hex}$ a la $FE9F_{hex}$, donde se almacenan hasta 40 sprites teniendo cada sprite 4 bytes asociados. Los dos primeros Bytes indican las coordenadas Y y X del sprite en ese orden, el byte numero 3 es el identificador del tile que se va a usar como sprite y el último de los bytes indica los atributos del sprite donde los bits 0-2 indican la paleta de la Game Boy Color, el bit 4 a la paleta de la Game Boy (0 paleta base, 1 paleta alternativa), el bit 5 al volteo horizontal, el bit 6 al volteo vertical y el bit 7 a la prioridad sobre los tiles.

Para poder dibujar un sprite correctamente, se deben escribir los datos necesarios referentes al sprite en el OAM durante un periodo de tiempo dentro del Vblank para lo que se podría volver a utilizar la información proporcionada en la dirección $FF41_{hex}$ tal y como se ha hecho para poder cargar los tiles en el BG map. Pero en este caso se va a hacer uso de la instrucción DMA (Direct Memory Acces) que es capaz de transferir un paquete de datos desde la ROM o la RAM al OAM durante el periodo de Vblank.

Para usar el DMA correctamente se deben cumplir una serie de requerimientos, siendo el primero de ellos, que el código que llame al DMA debe localizarse en la sección de memoria llamada HRAM, localizada en la misma sección de memoria que la pila. El segundo de los requerimientos es que se tiene que crear un buffer en la RAM con la misma cantidad de datos que se quieran mover al OAM. Es decir, si se quieren mover $9F_{hex}$ bytes de datos, se tiene que reservar esa misma cantidad de datos en la RAM, para ser copiados en el OAM. El crear el buffer sirve para que en cualquier momento de la ejecución, se puedan escribir los datos relacionados a los sprites en el mismo, para luego ser copiados al OAM.

Una vez creada la función encargada de usar el DMA para transferir los datos, solo queda llamarla durante el Vblank y para ello se emplean las anteriormente mencionadas interrupciones, en este caso la del propio Vblank.

6.4. Iteración 4: Mapas, Entidades y Físicas Iniciales

Hasta ahora se ha creado un proyecto funcional capaz de mostrar tiles y sprites por pantalla, y ahora toca completar un mapa, un jugador, un enemigo y la capacidad para moverse por el mapa respetando las colisiones del mismo.

6.4.1. Mapas

El primer paso es crear el mapa, y para esta fase del desarrollo se ha decidido crearlo del tamaño del propio BG map así como se le ha dedicado un archivo propio llamado maps.s. Para realizar esto es necesario tener en cuenta que los tiles que se están usando son de 16x16 px tal y como se ha visto en la iteración anterior por lo que hay que tener en cuenta dos tipos de mapas, el BG map, que va a ser la representación gráfica de 32x32tiles de 8x8 px y el Tilemap de 16x16 tiles que va a ser la representación lógica.

Esto provoca que cada tile del Tilemap sean 4 tiles en el BG map dibujados usando el procedimiento draw_tile descrito en la anterior iteración.

Teniendo esto en cuenta, lo primero es crear el Tilemap, que va a ser el que contenga todos los datos necesarios para la representación gráfica y las físicas. Este, esta compuesto por 16x16 bytes, representando cada uno de estos a un identificador de un tile, lo que permite al procedimiento draw_tile obtener la información que debe dibujar. A su vez este identificador se va a usar para distinguir qué tipo de interacción va a tener el jugador con el mismo, por ejemplo el identificador 00 hace referencia al tile del suelo mientras que el 01 lo hace con el tile de pared.

Al igual que los tiles, el Tilemap se crea inicialmente en la ROM aunque al iniciar el juego se copia a una dirección de la RAM. Esto se hace para que solo sea necesario cargar esta dirección de memoria a la hora de cargar los mapas.

Teniendo el Tilemap declarado, toca crear un procedimiento que sea capaz de, a partir de los datos del mismo, representar los diferentes tiles en el BG map. Este procedimiento se va a llamar draw_map. El funcionamiento de este comienza obteniendo el puntero al inicio del BG map y el puntero al inicio del Tilemap, una vez se tienen estos punteros, se obtiene el valor del Tilemap y se incrementa en 1 su puntero para que apunte al siguiente tile. Con el valor del Tilemap y el puntero al BG map se emplea draw_tile para dibujar los cuatro tiles de 8x8px y a continuación se incrementa en 2 el puntero al BGmap debido a la diferencia en el tamaño de los tiles del Tilemap y el BG map. El siguiente paso es comprobar si el BG map ha llegado al limite de la linea, si no se ha llegado al limite se vuelve a dibujar otro tile, pero de llegar al límite, se le suma al puntero del BG map lo suficiente para que se salte una linea, es decir si está en el límite de la primera linea, se incrementa el puntero hasta llegar al principio de la tercera linea. Este incremento se hace debido a la anterior mencionada diferencia en el tamaño de los tiles. Todo este proceso se repite hasta finalizar con el Tilemap.

Una vez siendo capaces de dibujar el mapa se procede a empezar con la lógica de lo que va a ser el jugador.

6.4.2. Entidades

Lo primero es crear el archivo player.s, que contiene toda la información referente al jugador, que en este caso son: dos bytes para las coordenadas en el mapa, dos bytes para las coordenadas en la vista y un byte para el identificador del tile que se va a usar como sprite. En este momento también se crea el archivo enemy.s con la misma información del jugador para usarla más adelante. Tanto el dibujado del jugador como el del enemigo se realizan de la misma forma y teniendo en cuenta que son sprites de 8x16px.

Hay que dibujar dos sprites para crear uno de 16x16 px. El primer paso es copiar al buffer de sprites los de la entidad relacionados con el mismo (coordenadas de vista y identificador del tile) así como un último byte de atributos, que se deja a 00 para esta primera mitad. Para la segunda mitad, se realiza el mismo procedimiento con la diferencia de que en el byte que indica la coordenada en x, hay que sumarle 8 y en el byte de los atributos ay que activar el bit 5 para que invierta el sprite en el eje x.

Ahora, toca comprobar el input de los botones de la cruceta para poder mover al ju-

gador en la dirección elegida. Para esto en el bucle infinito llamado game_loop declarado en la Iteración 2 se comprueba el valor de la dirección de memoria $FF00_{hex}$. El funcionamiento de esta dirección de memoria es un tanto especial, pero en resumidas cuentas, permite obtener el input de la cruceta escribiendo en esta dirección de memoria el valor EF_{hex} (en el caso de querer obtener el resto de los botones, se debe escribir el valor DF_{hex}). Una vez escrito el valor el la dirección$FF00_{hex}$, toca leer el valor de la misma, pues la consola habrá modificado los bits del 0 al 3 indicando las direcciones que se han pulsado, lo hace en el siguiente orden: derecha (bit 0), izquierda (bit 1), arriba (bit 2), abajo (bit 3).

Ya que se puede comprobar el input que realiza el jugador es el momento de realizar el movimiento por el mapa para lo que haremos uso de la View mencionada en la iteración pasada., que en este caso se usará para simular el movimiento del jugador por el mapa. El efecto que se busca es el de moverse a través de los tiles como si de un tablero se tratase con una suave transición entre los mismos, durante la cual no se puede realizar ninguna otra acción. Para conseguir este efecto hay que mover la View en intervalos de 16 píxeles por todo el BG map, y para esta necesidad están las direcciones de memora $FF42_{hex}$ y $FF43_{hex}$ que se encargan de la posición de la vista en los ejes Y y X respectivamente. De forma resumida, cada vez que se detecta un input se entra en un bucle que, durante 16 ciclos, desplaza la View en la dirección indicada por el input recibido. En el caso del jugador, no hay que realizar ningún ajuste en el sprite, pues siempre se mantiene en el centro de la pantalla, pero para los enemigos sí que hay que modificar el sprite, de hecho hay que aplicarle el mismo movimiento que se le aplica a la View pero en la dirección opuesta para simular su acercamiento o alejamiento al jugador.

6.4.3. Físicas Iniciales

El siguiente paso tras el movimiento es realizar el sistema de colisiones para el movimiento del jugador, para esto emplearemos las coordenadas en el mapa del jugador y el propio mapa. Debido a que el jugador se mueve en 8 direcciones, se deben hacer múltiples comprobaciones de colisiones con el mapa para saber si se puede mover o no. En estas colisiones iniciales, solo se comprueba si el tile hacia el que se quiere moverse es igual al tile de suelo, y en caso de ser diferente, se considera colisión. Dicho todo esto, el primer paso es comprobar las colisiones en el eje X, para lo que a la posición de jugador se le suma la dirección en X, si la casilla en la que cae es de suelo, se comprueba en el eje Y, y en el caso de que cada eje por separado no colisione, se comprueba en ambos ejes a la vez, para sacar la posición final. Esto se hace debido a que no esta permitido moverse en diagonal si hay una esquina de una pared de por medio. Una vez calculadas las colisiones con el mapa, hay que comprobarlas con el resto de entidades, que en este caso solo hay un enemigo. En este caso solo se comprueba la posición final del jugador, pues si que te puedes mover en diagonal a través de los enemigos.

Para finalizar, se comprueba si el jugador se encuentra en la misma casilla que el enemigo. De ser así, se elimina al enemigo para simular su derrota.

6.5. Iteración 5: Mapa con carga dinámica

En este punto se puede dibujar el contenido de un Tilemap que ocupa todo el BG map de la Game Boy, que son 16x16 tiles de 16x16 píxeles cada uno. Esto nos limita en el tamaño que pueden tener los mapas si se quieren cargar todos los tiles a la vez. Debido a que los mapas que se quieren representar son de 40x34 tiles, se deben realizar una serie de funciones que permitan cargar porciones de mapa conforme se va avanzando por el mismo.

El concepto tras este procedimiento es que, al igual que en el BG map existe el elemento View, en el Tilemap debe haber un elemento similar del mismo tamaño proporcional a la View, capaz de ser el punto de conexion entre el tilemap y el BGmap. A este concepto se le va a denominar como TView.

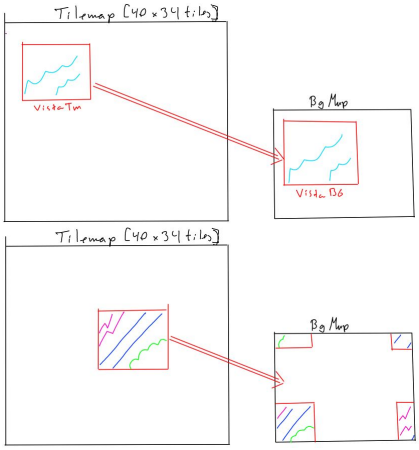


Figura 6.6.: Esquema del dibujado una porción del tilemap en la vista

Antes de empezar a crear los métodos necesarios, se deben tener en cuenta unas variables que van a ser necesarias a la hora de ejecutar los diferentes procedimientos. Estas variables son: Un puntero a la posición del Tilemap desde la que se va a empezar a dibujar (esquina superior izquierda), tres punteros que indiquen la posición de la vista en el BG map (esquina superior izquierda, esquina superior derecha y esquina inferior izquierda), el tamaño ancho del mapa y el alto del mapa .

El primero de los procedimientos que se van a crear es el método draw_screen, capaz de dibujar la porción del Tilemap que corresponde a la TView en la porción del BG map que corresponde a la View, es decir, es un procedimiento que dibuja toda la sección sobre la que se encuentre la View con la porción del Tilemap correspondiente. A la hora de realizar esta función hay que tener en cuenta que la View funciona de forma cíclica tal como se mencionó en la iteración 3, es decir, si la vista se pasa por la derecha del BGmap, vuelve a empezar por la izquierda, esto provoca que la vista pueda estar a

dividida hasta por cuatro zonas del BGmap (dos en X y dos en Y). Teniendo esto en cuenta, el dibujado se puede llegar a complicar si no se tiene en cuenta los límites del BG map, pero la base del dibujado es sencilla.

Primero se obtienen los punteros del comienzo del Tilemap y el de la esquina superior izquierda del BG map y con estos datos se empiezan a dibujar los tiles siguiendo el mismo procedimiento que con el anteriormente explicado draw_map, con las diferencias de que en vez de tener el ancho del BG map, se tiene el ancho de la vista. Si por el camino se alcanza algún límite del BG map, solo se debe corregir la dirección sobre la que se va a dibujar, tanto en X como en Y.

El problema radica en cómo saber si se está en uno de estos límites pues no se ve a simple vista En el caso del eje x el borde derecho se comprueba si la dirección acaba en 'xE' donde x es un número impar y en el borde izquierdo se hace comprobando con 'y0' donde y es un numero par. En el caso del eje y es más sencillo pues solo se debe comprobar si la dirección está, en el caso superior, dentro del intervalo [9800_{hex} -$981F_{\text{hex}}$] y en el caso inferior en el intervalo [$9BC0_{\text{hex}}$ - $9BDF_{\text{hex}}$]. Se debe destacar que todas estas correcciones están realizadas dentro de la premisa de que cada tile que se dibuja es de 16x16px, su se usan otras dimensiones de tiles, las comprobaciones pueden cambiar.

El siguiente método a realizar es uno capaz de dibujar columnas de tiles que se llamará draw_column. Este se usa para ir cargando los tiles del mapa a medida que el jugador se mueve por el eje horizontal. Las columnas tienen un tamaño dos tiles mayor al de la altura de la vista debido a que se el jugador se puede mover en diagonal y estos tiles extra evitan los posibles errores que puedan surgir. Esta columna se dibujará a la izquierda o a la derecha de la vista en función de la dirección que tenga el jugador y, dependiendo de dirección seleccionada, se usará el puntero al BG map de la esquina superior izquierda (se mueve a la izquierda) o el de la esquina superior derecha (se mueve a la derecha). Al igual que en el método draw_screen, se tienen que controlar los límites del BG map.

El último método, es muy similar al del dibujado de columnas, con la única diferencia de que en este caso son filas y se emplea a la hora de moverse sobre el eje vertical.

En la figura [6.7] se pude ver un esquema representativo de la carga dinámica de mapa por filas y columnas.

6.6. Iteración 6: Modelo ECS

Hasta ahora, todo el código que se ha ido programando ha sido con una mínima estructura, creada simplemente para probar funcionalidades, mecánicas y las capacidades de la Game Boy.

Por ello, este es el momento de estructurar todo el proyecto, y en este caso, se ha elegido el modelo ECS (Entity Component System) como base para el mismo, modificándolo para hacerlo más viable teniendo en cuanta las limitaciones de la consola.

Inicialmente, el modelo ECS se basa en tres factores, como su propio nombre indica:

- Entidades, cuyo principal objetivo es servir como modelo y estructura de datos.
- Componentes, son los datos en crudo de los que se componen las entidades.

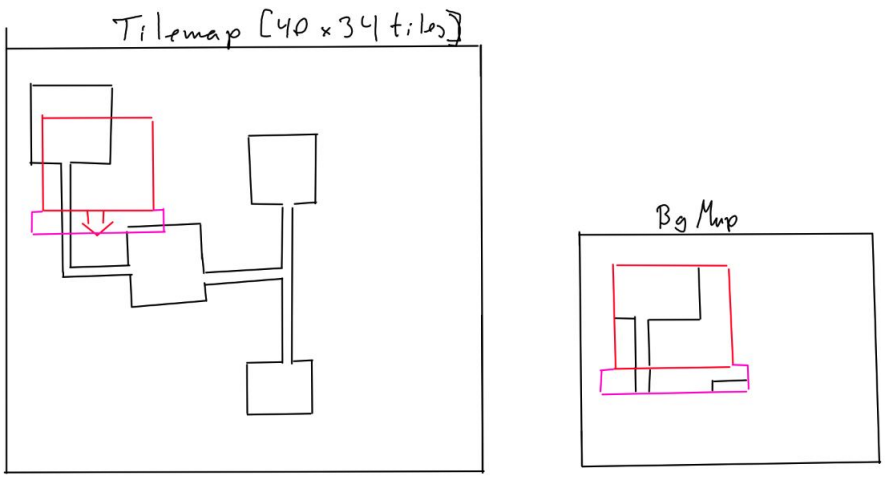


Figura 6.7.: Esquema de la carga dinámica del mapa en el eje Y

- Sistemas, funciones encargadas de recibir componentes y modificarlos en función de los datos recibidos.

Por ejemplo si se tiene una entidad jugador con los componentes de posición y de velocidad, y se le aplica un sistema de físicas de movimiento, este último modificará los componentes de posición teniendo en cuanta los de velocidad.

Pese a ser un modelo totalmente válido y usado a día de hoy, se ha optado por modificarlo para hacerlo más sencillo y ligero. La primera modificación es agrupar a los componentes y las entidades dentro de un mismo apartado, implicando que cada entidad distinta tendrá unos componentes propios y que no todos los sistemas funcionarán para todas las entidades. Los sistemas se mantendrán de la misma manera que en el original, la diferencia es que en este caso, reciben y modifican los componentes de ciertas entidades en concreto. El último cambio es el añadido de los Managers o Gestores, cuya función principal el organizar y gestionar los datos de las entidades, no modificarlos, esto con el propósito de desvincular a las propias entidades de los sistemas.

En cuanto al código, se ha tenido que reestructurar el sistema de carpetas por uno que muestre de forma más directa a qué hace referencia cada archivo. Hasta ahora, todo el código ha estado dentro de la carpeta src/ del proyecto así que se ha tenido que cambiar eso por una estructura de archivos que vaya acorde con lo representado en el diagrama. En este caso, dentro de src/ se han creado tres carpetas src/ent/, src/mod/ y src/sys/ haciendo referencia a los componentes del modelo ECS modificado, dejando la carpeta src/ base para aquellos archivos que no pertenezcan a ninguno de los componentes del modelo ECS, como el archivo main.s o el header.s. También se ha cambiado la nomenclatura de los diferentes archivos añadiéndoles un prefijo que indiquen el tipo que son, por ejemplo, todas las entidades tienen el prefijo 'entity_', mientras que los modelos y

los sistemas tienen 'manager_' y 'system_' respectivamente.

Comenzando con las entidades las entidades, se han eliminado los archivos player.s y enemy.s, y se ha creado un fichero genérico llamado entity_playable.s donde se establecen todos los componentes que comparten tanto jugador como enemigos. Estos son referentes a la posicion, al movimiento, al sprite que utilizan, a las estadísticas base (vida, ataque, defensa y magia) y a las estadísticas durante el juego, que en un inicio son las mismas que las base, pero pueden aumentar o disminuir dependiendo de la situación.

A partir de este entity_playable se generan dos entidades más, ya específicas para el jugador y los enemigos, y estas son entity_player.s y entity_enemy.s. Estas dos nuevas entidades utilizan todos los componentes definidos en entity_playable y añaden nuevos específicos para cada una, por ejemplo en el caso de entity_player, se añaden componentes para saber el equipamiento del jugador así como unas estadísticas base que serán modificadas por el propio equipamiento. En el caso de entity_enemy los componentes que se han añadido son referentes a la IA.

Al igual que se ha sustituido player.s por su equivalente en el modelo ECS, también se han creado nuevas entidades para poder adaptar el código existente al nuevo modelo. Por ejemplo, se ha creado una entity_camera, cuya función es agrupar todos los componentes necesarios para poder realizar la carga dinámica del mapa así como se ha creado una entity_map que sirve como modelo para almacenar los diferentes Tilemaps que se quieran implementar.

Inicialmente, se han creado tres principales managers, siendo el principal manager_game.s. Este se encarga de agrupar y gestionar el resto de managers y los diferentes sistemas del juego, es en esencia, el núcleo del juego donde van a parar todas las funcionalidades. El resto de managers son manager_playable, que se encarga de gestionar la entity_player y un vector de entity_enemy para los diferentes enemimgos del nivel, y manager_level, que se encarga de gestionar tanto la entity_camera como el entity_map.

Por último, toca hablar de los sistemas. Estos se han dividido dependiendo de las funcionalidades que realizan, por ejemplo, el dibujado de sprites, tiles y el propio mapa se ha puesto en system_render, mientras que el movimiento y las colisiones se han puesto en system_physics. Como estos dos sistemas, se han creado otros tales como system_user, encargado de la interacción con el usuario y system_level, encargado de todo lo relacionado con la generación de niveles. Estos no son los únicos sistemas del proyecto pues se añadirán más conforme se vayan necesitando.

6.7. Iteración 7: Tiles Avanzados y Físicas Finales

Ahora y con el nuevo modelo de estructuración de datos es el momento de mejorar las funcionalidades del código realizado hasta ahora, empezando por el dibujado de tiles y sprites. Hasta ahora, tanto los tiles como los sprites se almacenaban de forma secuencial en mismo bloque de la memoria de vídeo, para ser mas exactos, en la dirección $[8000_{\text{hex}}$.

6.7.1. Tiles Avanzados

Para diferenciar las zonas de sprites y de tiles en la memoria de vídeo se ha optado por establecer el bit 4 de la dirección [$FF40_{hex}$ a cero, lo que provoca que los tiles del fondo se lean de la dirección [8800_{hex} a la [$97F0_{hex}$, dejando así las direcciones de la [8000_{hex} a las [$87FF_{hex}$ libres para el único uso de los sprites. De la misma forma, en la zona reservada para los tiles se han establecido dos grupos, uno destinado para los tiles que conforman el tileset del nivel del juego ([9000_{hex} - [$97FF_{hex}$) y otro destinado a los tiles del HUD y tiles auxiliares ([8800_{hex} - [$8FFF_{hex}$).

Otra mejora que se ha realizado sobre los tiles y los sprites es la creación de unos índices que permiten, mediante un ID, acceder a la información de un tile o sprite formado por tiles más pequeños. Esto se debe a que en la Game Boy los tiles son de 8 x 8 px y el juego está diseñado con tiles de 16 x 16 px en mente, por esto es necesario un índice que agrupe cuatro tiles pequeños diferentes para generar uno grande. En cuanto a los sprites, los índices son similares, pero en vez de hacer referencia a 4 tiles, solo se referencia a 2 de estos (debido a que los sprites usan un formato de 8 x 16 píxeles) y, entre tiles, se intercalan los atributos para el dibujado. En la figura [6.8], se puede ver de forma más esquemática el funcionamiento de los índices.

Debido a este nuevo método de representación de tiles y sprites, es necesario modificar el procedimiento draw_tile, que hasta ahora,a partir de un puntero al BGmap y una identificación de un tile, dibujaba el mismo tile cuatro veces para representar un tile de 16x16 px. La principal modificación a este método es que ahora en vez de recibir una identificación de un tile, recibe una posición dentro del índice de tiles, la cual contiene las identificaciones de los cuatro tiles de 8x8px que conforman el nuevo tile de 16x16px. También se han modificado todos los procedimientos relacionados con sprites de una forma similar.

6.7.2. Físicas Finales

Hasta ahora, se ha creado un detector de colisiones, pero no un sistema de físicas que mediante la detección de estas colisiones actúe en consecuencia. Por eso mismo, se va ha modificar la función que detecta las colisiones para realizar un sistema de físicas más acorde a lo que se espera del juego.

El primer paso son las colisiones con los tiles del escenario, que se mantiene de forma similar a como se ha hecho hasta ahora, siendo la principal diferencia la capacidad de deslizarse por las paredes en caso de moverse en diagonal hacia una de las mismas. Otro cambio realizado es que en el caso de que el tile al que se mueva el jugador sea un tile especial (Escaleras, Objetos, Dinero), se tiene en cuenta y se almacena para un futuro tratamiento.

El siguiente aspecto a tener en cuenta, son el resto de las entidades, aquí se incluyen tanto al jugador como al resto de los PNJs debido a que el método debe poder funcionar para cualquier entidad, con la única diferencia de que los PNJs no pueden interactuar con los tiles especiales anteriormente mencionados. Si a la hora de moverse se detecta colisión con alguna de las entidades, se interpreta como la colisión con un tile sólido y

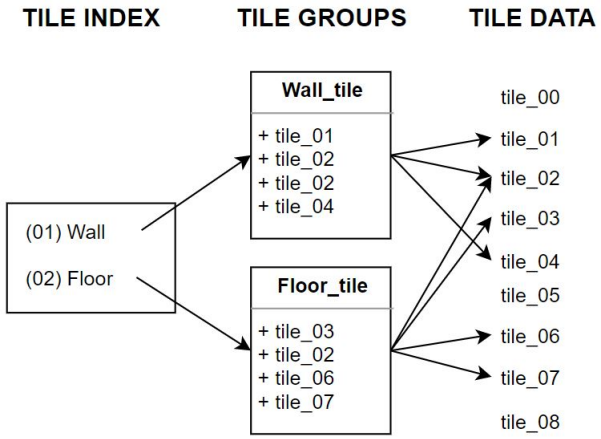


Figura 6.8.: Esquema de los índices de division de los tiles

se impide el movimiento, la diferencia es que solo comprueba la colisión en la posición final, lo que implica poder moverse en diagonal alrededor de una entidad.

La última de las comprobaciones que se realiza en las colisiones es con los ya mencionados tiles especiales, que pueden ser de dos tipos: las escaleras, que permiten el paso al siguiente nivel, y los objetos. En función de con qué se colisione, el juego responderá de una forma u otra, por ejemplo, si se colisiona con las escaleras, se indicará que el jugador ha completado el nivel.

El último añadido relacionado con las físicas es la capacidad de orientar la dirección del jugador sin necesidad de moverse manteniendo pulsado el botón B de la consola. Esto permite atacar a enemigos que estén en por detrás del jugador sin la necesidad de moverse. Para realizar esta acción, se comprueba se se está pulsando el botón B, acto seguido se comprueba si se está pulsando en alguna dirección de la cruceta, si es así, se cambia únicamente la dirección del jugador, sin movimiento.

6.8. Iteración 8: Enemigos, IA y Combate

6.8.1. Enemigos

Es hora de centrarse en los enemigos y sus comportamientos, y para ello lo primero es ser capaz de crear enemigos, por lo que el primer paso es crear un método que sea capaz de añadir enemigos a un vector. Todos los procedimientos que se explican a continuación forman parte del archivo manager_playable. El vector de enemigos puede almacenar una cantidad máxima de los mismos por lo que nunca va a poder haber más de 6 enemigos activos a la vez. A la hora de añadir un enemigo al vector, se debe pasar un ID de enemigo

lo que va a hacer que el método obtenga los datos necesarios para generar ese tipo de enemigos en concreto (Vida, ataque, defensa, etc). Una vez con los datos necesarios, el método guardará de forma ordenada dichos datos en el vector de enemigos a la vez que le asignará una dirección para dibujar su sprite.

Una vez siendo capaces de crear un enemigo, toca poder eliminar un enemigo del vector. En principio, es una tarea sencilla pues simplemente es acceder al comienzo del enemigo que se quiere eliminar en el vector, reiniciar todos sus valores a cero y restar uno en la cantidad de enemigos que hay disponibles, y eso sería suficiente si se tratara del último enemigo del vector. En el caso de que el enemigo que se quiera eliminar esté situado a la mitad del vector, toca reorganizar el mismo, situando al último enemigo del vector en la posición del que va a ser eliminado, obteniendo la dirección de dibujado para evitar posibles errores. Una vez hecha la reorganización, toca eliminar ahora si, el último enemigo del vector.

El siguiente paso a la hora de generar un enemigo en el mundo es asignarle una posición en el mapa y para ello se ha creado un procedimiento que le asigna aleatoriamente una coordenada X y una coordenada Y al enemigo dentro de las limitaciones del tamaño del mapa. Si las coordenadas caen en una casilla con un tile válido, donde no hayan otras entidades y que esté fuera del alcance de la vista del jugador, se considera un a posición válida y le asigna al enemigo.

6.8.2. Inteligencia Artificial

Hasta ahora se tienen enemigos que se generan y se destruyen en un mapa, pero no interactúan de forma directa con el jugador, por esto mismo se va a emplear un árbol de decisiones para que los enemigos tengan diferentes comportamientos en función de lo que suceda a su alrededor. Éste es un árbol binario, es decir, ante una situación puede tomar dos decisiones en función de si se cumple la condición o no. Por poner un ejemplo, si en enemigo tiene al jugador a la distancia de ataque le atacará, en caso de que el jugador esté lejos, optará por moverse. El árbol de decisiones se puede ver en la figura [6.9]

Si bien el árbol está claro, para que funcione correctamente, se deben añadir una serie de elementos a la lógica del juego, y estos son los conceptos de Sala y de Salida, añadidos a la lógica del juego como entity_room. Estos elementos permiten a los enemigos diferenciar si se encuentran en una sala o en un pasillo, y también permiten saber dentro de una sala, las diferentes opciones de movimiento que tienen, en la figura [6.10] se puede ver un esquema que representan a las salas y a las salidas. Las salas cuentan con un par de coordenadas X e Y, con unas dimensiones Ancho y Alto y con un vector de un máximo de cuatro salidas. Las dimensiones de las salas son más grandes de lo que visualmente se puede apreciar para poder acomodar correctamente a las salidas. Las salidas sin embargo solo cuentan con un par de coordenadas X e Y, que marcan los diferentes puntos de salida que tiene una sala. Al salir de una sala por una salida, el enemigo pasa a interpretar que está en un pasillo hasta que se encuentra con otra sala.

Para implementar toda la lógica representada en el árbol binario, se ha decidido crear el archivo system_IA. El procedimiento encargado de ejecutar dicha lógica se llama choose_IA_action. En este se van recorriendo todas las condiciones plasmadas en las diferentes

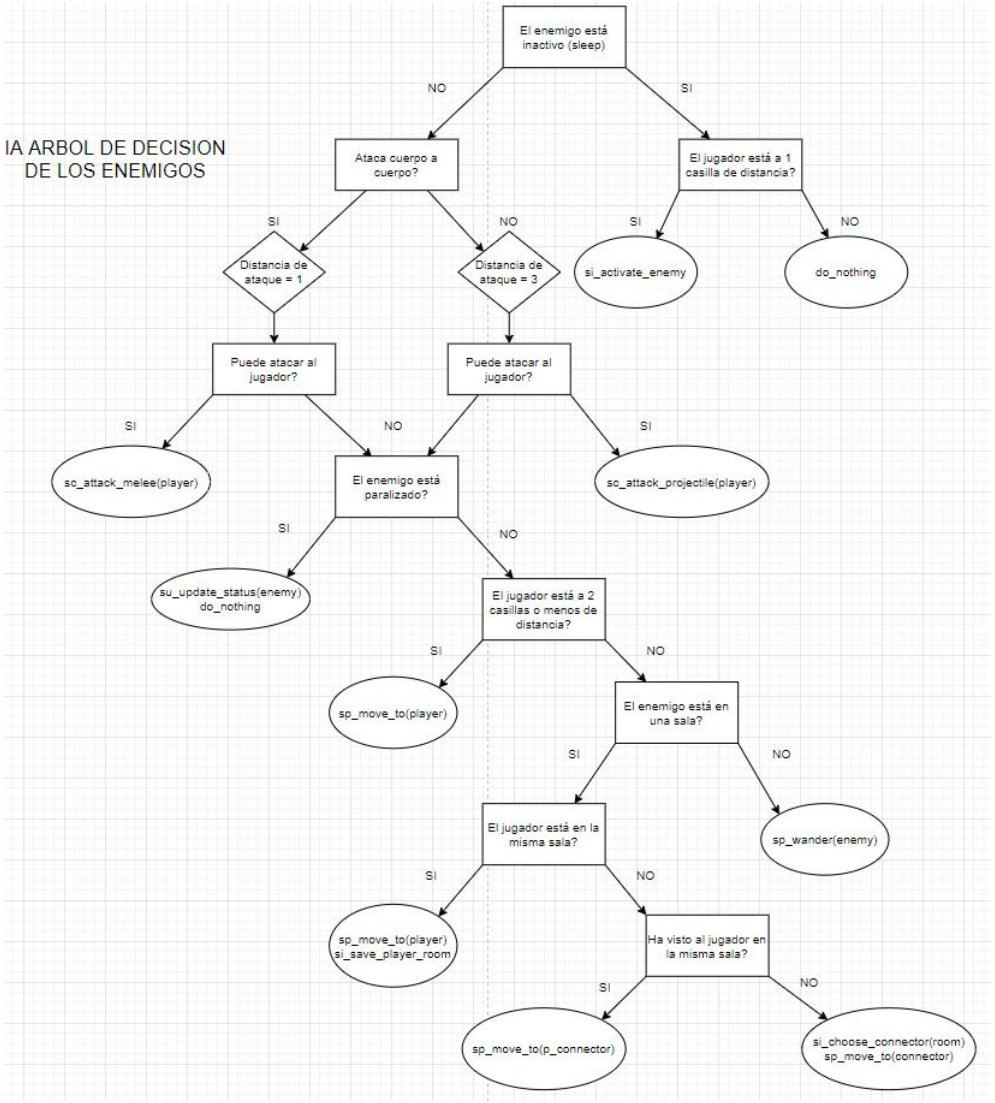


Figura 6.9.: Diagrama del Árbol de Decisiones

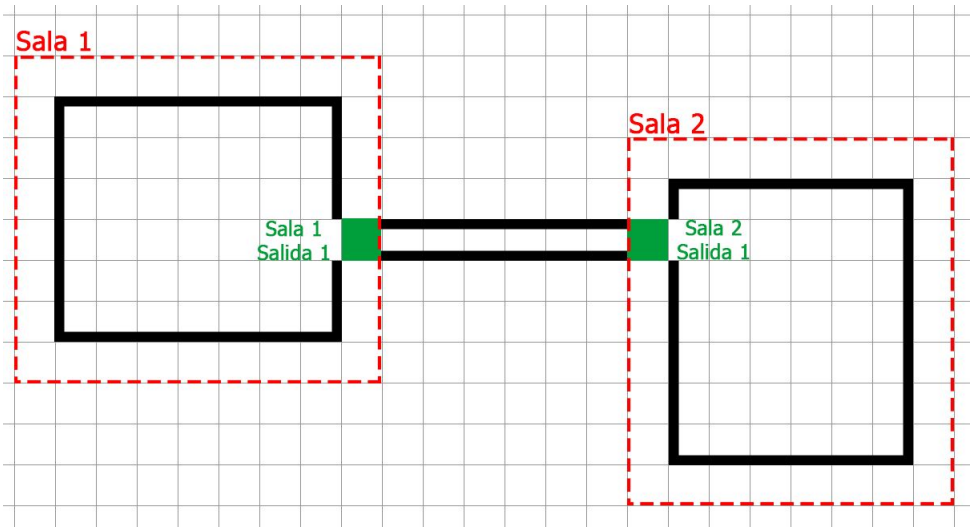


Figura 6.10.: Esquema de las salas y las salidas del mapa

ramas árbol y, en el caso de llegar a una de la hojas, ejecuta el procedimiento pertinente.

Ahora que está la lógica clara vamos a los dos procedimientos que permiten el movimiento en el enemigo, comenzando por el más sencillo de los dos, wander. Este método se usa únicamente en los pasillos, debido a su naturaleza. Su funcionamiento consiste en avanzar en una dirección pseudoaleatoria con la única condición de no poder moverse hacia la posición de anterior, lo que lo hace perfecto para moverse por los pasillos, pues si no se puede mover hacia atrás solo queda seguir el pasillo y en caso de alcanzar un cruce, la elección pseudoaleatoria escogerá una de las rutas de la intersección. La única forma de volver hacia atrás es que no se pueda mover en ninguna otra dirección, lo que permite que si se alcanza un punto muerto, pueda volver sobre sus pasos.

El último método es el move_to. Este método se usa en las salas y cuando el jugador está a una distancia de dos casillas o menos del enemigo. Permite al enemigo moverse hacia un objetivo en concreto, ya sea el propio jugador o una de las salidas de la sala en la que se encuentra y funciona obteniendo la dirección a la que se debe mover para alcanzar el objetivo y aplicando la misma a la hora de moverse. Debido a que en las salas no hay obstáculos, el enemigo solo debe avanzar de forma directa hacia el objetivo lo que simplifica los cálculos.

6.8.3. Combate

Ahora que los enemigos tienen movimiento toca centrarse en el combate cuerpo a cuerpo. Para empezar, tanto los enemigos como el jugador usan la misma fórmula para calcular el daño, y esta es:

$$D = A - \lfloor D/2 \rfloor \tag{6.1}$$

$$R = \begin{cases} 1 & \text{si } D < 1 \\ D & \text{si } D \geq 1 \end{cases}$$

En esta formula A representa la estadística de ataque del que efectúa el golpe mientras que D representa la estadística de defensa de quien lo recibe. El objetivo de esta formula es que por cada dos puntos de defensa se puede negar un punto de ataque, siendo el daño mínimo 1.

Para la realización de todos los procedimientos relacionados con el combate, se ha decidido crear el archivo system_combat. Los procedimientos contenidos en este archivo son:

- check_attack_melee. Este procedimiento se encarga de comprobar si el ataque puede ser efectuado, es decir, comprueba que en la dirección a la que se ataca no haya ningún obstáculo. Esto lo hace de forma similar al funcionamiento de las físicas respecto al escenario.

- attack_melee. Una vez se ha comprobado que en la dirección a la que se ataca no hay ningún obstáculo, se procede a comprobar si el ataque impacta con alguna entidad. Esto se hace obteniendo la posición de la entidad atacante a la que se le suma la dirección del ataque, acto seguido se comprueba si esa posición coincide con alguna de las posiciones de las entidades del juego.

- calculate_damage. Este es el procedimiento encargado de calcular el daño del ataque y adapta de forma directa la fórmula mostrada anteriormente.

6.9. Iteración 9: HUD y Menús

6.9.1. HUD

Para poder representar toda la información necesaria sobre el jugador, es necesaria la creación de un nuevo elemento gráfico, el HUD (Head-Up Display). En este se van a mostrar datos tales como la vida del jugador, la cantidad de maná que le queda y el piso de la mazmorra en el que se encuentra.

El primer paso es cargar en memoria todos los tiles que van a componer el HUD. Tal y como se ha mencionado anteriormente, la VRAM destinada a los tiles se ha dividido en dos secciones, de [9000_{hex} - $97FF_{\text{hex}}$], que va destinado al tileset del nivel y de [8800_{hex} - $8FFF_{\text{hex}}$], que va destinado a tiles auxiliares y al HUD. En esta la dirección 8800_{hex} se cargarán los siguientes tiles en este orden: tile de fondo, tiles numéricos del 0 al 9, y tiles decorativos o de información. El orden será importante más adelante para la correcta representación de los datos.

Una vez los tiles están cargados en memoria toca dibujar el HUD base usando un nuevo procedimiento llamado draw_hud, situado en el archivo system_render. Este dibujado se

va a realizar sobre la Window. La Window es muy similar al BG map en cuanto a la hora de mostrar los tiles, la diferencia radica en que por defecto no es visible y de serlo, se muestra por encima del BG map. Para activar el uso de la Window, se debe de activar el bit 5 del control LCD ($FF40_{hex}$), también es recomendable hacer que la Window dibuje sobre las direcciones [$9C00_{hex}$ - $9FFF_{hex}$] pues por defecto dibuja sobre las mismas direcciones que el BGmap, para hacer esto se debe activar el bit 6 del control LCD.

El dibujado del HUD en sí es sencillo. Partiendo de la posición $9C00_{hex}$ se empieza cargando el tile que representa el fondo del HUD un total de 20 veces, esto genera una línea uniforme del ancho de la pantalla. Una vez hecho esto, se cargan los tiles numéricos y de información en sus respectivas posiciones. Aunque los valores de los tiles numéricos no coincidan con los valores actuales del jugador, estos sirven como base para comprobar el resultado y más adelante se modificarán para que puedan mostrar el valor que les corresponde.

En este momento ya se pueden dibujar los tiles sobre la Window, pero aunque esta esté configurada y activa, no se va a mostrar nada pues queda establecer su posición respecto a la View. En este aspecto es muy similar a los sprites, en las direcciones $FF4A_{hex}$ y $FF4B_{hex}$ se encuentran la posición Y y X de la Window respecto a la view. Estas posiciones indican a partir de que posición de la view se van a empezar a mostrar los tiles de la Window, limitando las opciones que se tienen para dibujar el HUD, pues si se pone la Window arriba, esta misma tapará el resto del BG map. Por eso mismo se ha optado por poner el HUD en la parte inferior de la pantalla.

Tras realizar todos los pasos, el HUD base se muestra correctamente con una excepción, los Sprites se muestran por encima de este y pueden llegar a tapar información relevante. Para solucionar eso hay que volver a usar el control LCD junto con la interrupción LCD-Stat. El primer paso de este proceso es activar la interrupción de LCD-Stat al comienzo del main, para ello se activa el bit 1 de [$FFFF_{hex}$]. EL siguiente paso es establecer el LYC (LCDC Y-coordinate Compate) [$FF45_{hex}$] a la linea de dibujado que se quiere, es decir a la coordenada Y de la View que coincide con el comienzo del HUD. El LYC se compara con el LY (LCDC Y-coordinate) del dibujado continuamente. Mientras que LYC se mantiene estable en el valor establecido, el LY varia de valor en función de la linea de dibujado en la que esté la consola. Una vez establecido el LYC, hay que activar el bit 6 del Estado del LCD ($FF41_{hex}$) para activar la comprobación entre LYC y LY, saltando la interrupción LCD-Stat cuando ambos coincidan. El ultimo paso es que cuando salte la interrupción LCD-Stat, se activa un procedimiento que desactiva el dibujado de sprites modificando los bit 1 y 2 del control LCD. Para reactivar los sprites, se modifican los mismos bits del control LCD pero en la interrupción del Vblank.

Para acabar con el HUD queda realizar un último proceso, mostrar los datos del jugador correctamente. El primero paso realizado para lograr esto es crear una nueva entidad, entity_hud. El proposito de esta entidad es el de almacenar todos los datos referentes al jugador que se quieran mostrar, ya sea la vida, el ataque o el nivel. La principal característica de esta entidad es que guarda los valores en decimal, con el objetivo de facilitar el dibujado. Cada dígito en decimal tiene un byte asociado, por lo que una estadística como la vida cuyo valor máximo es 255 tendrá asociados 3 bytes,

uno para las unidades, otro para las decenas y un último para las centenas.

Obtener los valores decimales a partir de un número en hexadecimal no es trivial, por eso se ha generado el procedimiento hex_2_dec, que es capaz de convertir un byte en hexadecimal a un numero decimal. Dado que el valor máximo que puede alcanzar un byte en hexadecimal es el 255, el primer paso es comprobar si el número es mayor de 200, de esta comprobación se puede obtener el valor las centenas, restando dicho valor al byte original. Acto seguido se comprueba si es mayor de 100, para obtener nuevamente el valor de las centenas en caso de que el número fuera menor de 200. Una vez obtenido el valor de las centenas, se analizan los bits 0 al 6 del byte resultante. Por cada bit que esté con el valor 1, se realiza una suma donde el valor viene determinado por el bit en cuestión, por ejemplo el bit 1 sumará 1_{hex} mientras que el bit 6 sumará 64_{hex}. A la hora de realizar esta suma es importante emplear la instrucción daa para que el proceso de como resultado un numero decimal. El resultado de esta operación da por un lado un byte con el valor 1_{hex} o 2_{hex}, que representa el dígito de las centenas y por otro lado un byte donde cada uno de los nibbles representa el contenido de las decenas y las unidades en decimal.

El paso final para completar el HUD es representar los tiles numéricos correctamente. Para lograrlo, e ha generado un par de procedimientos llamado draw_number_3 y draw_number_6. Ambos procedimientos funcionan de la misma forma y la única diferencia entre ellos es la cantidad máxima de dígitos que pueden mostrar. El funcionamiento general es que, a partir los dígitos en decimal, un puntero a la zona del BG map donde se quiere dibujar y un offset, se dibujan uno a uno los tiles correspondientes de cada dígito. El único apartado complicado de todo esto es saber qué tile corresponde a cada dígito, y para eso está el offset. Debido a que al cargar los tiles en la VRAM se hizo de forma ordenada, el offset sirve para sumarlo a cada uno de los dígitos decimales y así sacar el tile que le corresponde.

6.9.2. Menús

Una vez teminado el HUD, el siguiente objetivo es el de generar los diferentes menús del juego, tales como el menú de inicio, el menú de pausa o el menú de objetos.

Hasta ahora, solo se ha tenido un único estado del juego, el game_loop. Para poder crear los diferentes menús se deben crear más estados de juego, uno por cada menú. Cada estado de juego tiene un bucle propio para gestionar los inputs, el renderizado y la lógica interna. Todos los estados del juego se gestionan en manager_game, pero es en el main donde se decide cual de ellos va a ser el activo. A continuación se van describir todos los estados en los que puede estar el juego:

- Menú de inicio. El estado main_menu es el primero en activarse una vez se enciende el juego. En este se muestra la pantalla del título cargada mediante el uso del procedimiento draw_screen descrito en la iteración 5 [6.5]. La única comprobación que se hace en este estado es la del input, pues en cuanto se pulsa un botón de acción, se cambia al estado del game_loop.

- Ciclo del juego. El estado game_loop es el principal a la hora de jugar. En este se gestiona la IA, el movimiento del jugador y el resto de mecánicas jugables. Desde este estado, se puede cambiar al main_menu si se agotan los puntos de vida o al pause_menu si se pulsa el botón start.
- Menú de pausa. El estado pause_menu solo se puede activar desde el game_loop. En este estado se muestran las estadísticas del jugador como el ataque, la defensa o el nivel. También se gestiona el input del jugador para actualizar el estado de un selector. En función de la selección, cuando se pulsa el botón A se avanzará a otro estado del juego, ya sea el game_loop o el item_menu. Si se pulsa el botón B o el botón start, se cargara el estado del game_loop.
- Menú de objetos. El estado Item_menu es el principal responsable de la gestión de los objetos, pero por ahora lo único que se muestra es el fondo del menú, pues los objetos no están implementados todavía. Si se pulsa el botón B, se voverá al estado pause_menu.

Aunque no se ha mencionado durante la descripción de los estados, para cada uno de estos se ha creado un procedimiento init que inicializa todos los recursos necesarios para la correcta representación de los diferentes estados.

6.10. Iteración 10: Objetos y Equipo

6.10.1. Objetos

Hasta ahora el jugador ha sido capaz de colisionar con tiles que representan los diferentes objetos que se pueden encontrar por el mapa, pero en realidad los objetos como entidad no han estado presentes. Por eso, el primer paso para introducir los objetos en el juego es creando la entidad entity_item. Esta entidad contiene el ID del objeto que representa, unas coordenadas X e Y el ID del tile que va a usar para representarse en el mapa y la cantidad del objeto.

Como se puede observar, la entidad de los objetos no tiene ninguna referencia al tipo de objeto que es más allá del ID. Esto se debe a que, al igual que se hace con los tiles del mapa, el ID del sprite hace referencia a una posición dentro de un índice de objetos, de donde se puede obtener toda la información relevante del objeto. Esta información contiene al menos el nombre del objeto y una descripción, el resto de información depende del tipo de objeto. En el caso de este juego, hay cuatro tipos de objeto: armas, escudos, bastones mágicos y consumibles.

Para tener un control de todos los objetos distribuidos por el mapa, se ha decidido crear un vector de objetos. También se ha creado un vector de objetos para el jugador, con la diferencia de que este último solo almacena los ID de los objetos.

Para poder recoger un objeto del suelo, en cuanto el jugador se mueve hacia una casilla donde hay representado un objeto, se busca en el vector de objetos del mapa el objeto cuya posición coincida con la de la casilla. Una vez encontrado, se reemplaza la casilla del objeto por una de suelo y se añade el ID del objeto al vector del jugador.

6.10.2. Equipo

A continuación se va a tratar el uso de los objetos por parte del jugador, pero antes hay que retomar el item_menu descrito en la iteración anterior.

Ahora que el jugador cuenta con un vector de objetos, estos ya se pueden mostrar en el item_menu. El primer paso para lograr esto es crear un procedimiento capaz de representar texto por pantalla, draw_text.

Hay que tener en cuenta que el ensamblador RGBASM es capaz de codificar un string de texto. Esto lo hace convirtiendo cada letra del string a su código ASCII en hexadecimal, por lo que al escribir el string 'Hola' en el código, este se guardará como 48_{hex} $6F_{\text{hex}}$ $6C_{\text{hex}}$ 61_{hex}.

Teniendo esto en cuenta, el procedimiento draw_text funciona de una forma similar a procedimientos como draw_number_3, con la diferencia de que cargar todo el abecedario en la VRAM no es muy conveniente. Por eso draw_text lo primero que hace es cargar los tiles de las letras que necesita para representar el texto en VRAM, y luego usa esos tiles a la hora de mostrar el texto por la pantalla.

Gracias a draw_text ya se pueden representar los nombres de los objetos en el item_menu, pero no se pueden mostrar los nombres de los 20 objetos que puede tener el jugador a la vez. Para solucionar esto, los 20 objetos se dividen en cuatro grupos de 5 objetos cada uno. Para ir mostrando los diferentes grupos, se ha implementado un sistema de páginas en item_menu. Cada vez que que se pulsa a izquierda o derecha la cruceta se cambia de página, mostrando los 4 objetos siguientes o anteriores.

Dentro de cada página, se puede pulsar en la cruceta las direcciones arriba y abajo para seleccionar un objeto de dicha página. En la parte inferior del menú se muestra un texto con la descripción del objeto seleccionado, y en el caso de pulsar el botón A, se muestra un marco de texto con diferentes opciones en función del objeto:

- Use: Esta opción solo aparece si el objeto seleccionado es un bastón mágico o un consumible. En caso de haber seleccionado un objeto consumible, este se eliminará del vector de objetos del jugador y curará la cantidad de salud y magia especificada en los atributos del objeto.

- Equip: Esta opción solo aparece para armas o escudos. Permite equipar un arma y un escudo al jugador, aumentando las estadísticas de vida máxima, maná máximo, ataque o defensa en la medida especificada en cada objeto.

- Unequip: Esta opción solo aparece para armas o escudos equipados. Permite quitar el equipamiento, perdiendo en el proceso las características ganadas por el objeto.

- Delete: Elimina el objeto del vector de objetos del jugador

- Exit: Permite seleccionar otro objeto.

6.11. Iteración 11: Generación Procedimental

Hasta ahora todos los elementos de cada nivel del juego se han estado poniendo a mano uno a uno lo que supone un gasto de tiempo y recursos muy elevado teniendo en cuenta que el juego consiste en explorar una mazmorra. Para solucionar este problema se ha optado por crear un generador de niveles procedimental capaz de situar todos los elementos necesarios para poder completar el nivel.

El generador procedimental que se va a describir más adelante está basado en los trabajos realizados por el usuario fsd66 [5] y Hector Pulido [6].

El generador realizado por Hector Pulido es el mas sencillo conceptualmente. Inicialmente se genera una matriz cuadrada que se considera el mapa. Sobre esta matriz se genera un numero aleatorio de salas teniendo en cuenta que no se pueden solapar. A continuación se seleccionan puntos aleatorios dentro de cada sala como salidas y por último se unen dichos puntos con pasillos para comunicar las diferentes habitaciones.

El siguiente generador es el creado por el usuario fsd66 y se basa principalmente en un BSP (Binary Space Partition). Al igual que en el anterior ejemplo, se parte de una matriz cuadrada que se plantea como mapa. El siguiente paso es realizar un partición binaria que divide la matriz principal en dos secciones sobre las que se realizarán sucesivas particiones hasta alcanzar un numero de iteraciones establecido. El siguiente paso es, dentro de cada parcela resultante, generar una sala en una posicion y con un tamaño aleatorio, y una vez hecho, solo queda unir las habitaciones con pasillos por parejas. El emplear un BSP asegura que todas las secciones puedan unirse por parejas de la forma inversa a la que se han generado mediante particiones binarias.

En el caso de este proyecto, la generación del mapa comienza con una matriz de un tamaño definido sobre la que se generan de 3 a 5 habitaciones rectangulares con tamaños y posiciones variables. Hay que tener en cuenta que las habitaciones no pueden solaparse, deben tener un mínimo de una casilla de distancia entre estas y que, tal y como se ha definido en el apartado de IA y Enemigos, las habitaciones ocupan un espacio extra en cada dirección para controlar las salidas, lo que provoca que una sala de 5x5 tiles solo tenga un espacio de 3x3 tiles de suelo. También hay que controlar que las salas generadas no estén demasiado cerca de los bordes del mapa, pues se mostrarían por pantalla zonas de memoria que no pertenecen al mapa, por esto se han establecido márgenes de 6 tiles a izquierda y derecha y de 5 tiles arriba y abajo.

Estos primeros pasos están basados en la aproximación de Hecto Pulido, añadiendo restricciones para adaptarlo al hardware de la Game Boy.

Para conectar las diferentes salas generadas, se efectúa una operación similar a la realizada en el caso del usuario fsd66, se unen las salas por parejas de forma iterativa hasta que todas estén unidas. En el caso de este proyecto, empezando por la primera sala, se busca la más cercana de entre el resto usando el siguiente cálculo: $D = Dx + Dy$. Donde D es la distancia total, Dx es la distancia en el eje x y Dy es la distancia en el eje y. Una vez se tiene la sala más cercana se comprueba si ya se ha conectado a esta misma sala, es decir, si la sala 0 se ha conectado con la 1, la sala 1 ya no puede considerar a la 0 como la más cercana. Esto se hace para evitar bucles entre salas y garantizar que cada sala se conecte con al menos una. Una vez se ha seleccionado una sala como la más

cercana, se genera una salida en cada una de las salas que se quiere conectar y se une con un pasillo, tal y como se muestra en el esquema [6.11]. Este proceso se repite con cada una de las salas generadas inicialmente.

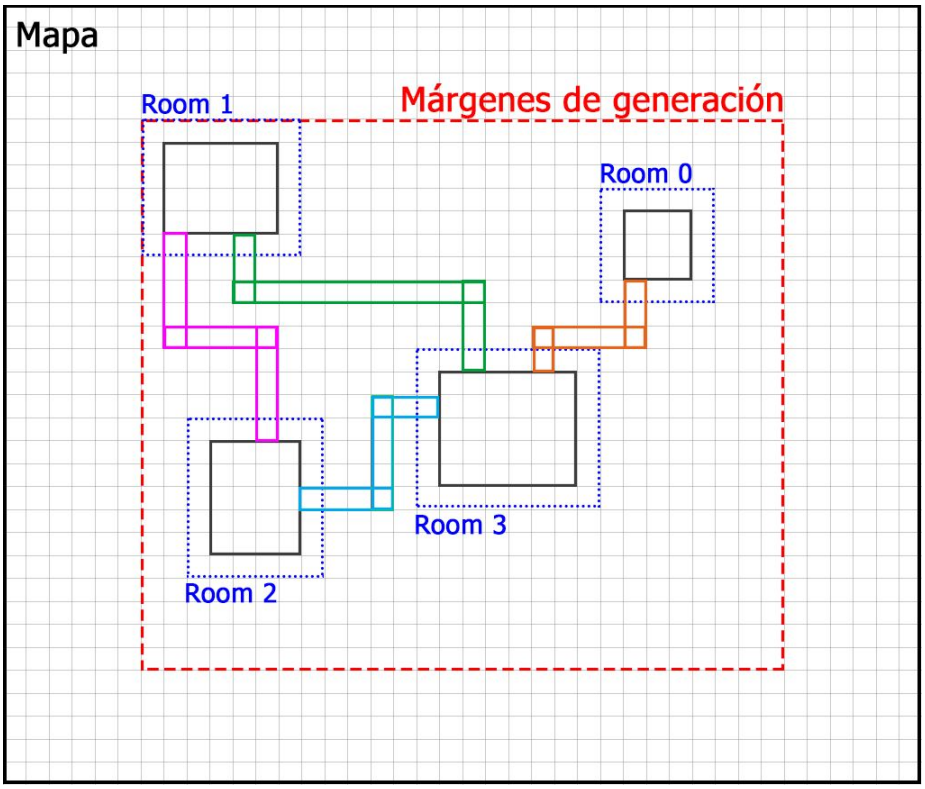


Figura 6.11.: Esquema que muestra la generación procedimental del proyecto.

Un posible error que se puede dar con esta configuración es que cuando se genera un número igual o superior a 6 salas, pueden generarse grupos de salas sin conexión entre los mismos, lo que podría generar un nivel que no se puede completar. Para solucionar este caso se puede usar un vector de conectividad. Este inicialmente contiene un ID de cada sala en el mapa. Cada vez que se realiza una conexión entre dos salas, estas se eliminan del vector y en su lugar se añade al vector un ID que representa la propia conexión entre las salas. Si al acabar de conectar todas las salas hay más de un elemento en el vector de conexiones, significa que se han formado varios agrupaciones de salas que no se conectan entre ellas. Para solucionar este caso, solo hay que unir alguna de las salas de los grupos que están separados.

El último paso para terminar de generar el mapa, es posicionar los objetos dentro de las salas, generar las escaleras y posicionar al jugador dentro de una sala.

7. Conclusiones

Durante todo el proyecto se han superado diferentes fases de desarrollo, unas más gratificantes, otras más tediosas e incluso algunas que se pensaban inabarcables. Durante este periodo también ha ocurrido confinamiento provocado por el COVID-19, que también ha hecho mella en el rendimiento debido a las condiciones de la zona de trabajo. Pero pese a todos los obstáculos, con mucho esfuerzo y horas de dedicación se ha conseguido avanzar hasta el punto en el que por fin se ha completado el desarrollo de Project Rogue.

Ha sido una experiencia muy enriquecedora tanto a nivel de conocimientos como a nivel personal donde se ha alcanzado el objetivo principal, desarrollar un videojuego para la Game Boy. En cuanto al resto de objetivos, a continuación se muestra un listado con cada uno de ellos:

- Diseño del videojuego Project Rogue. Se ha cumplido el objetivo propuesto de diseñar un videojuego del género roguelike con los elementos representativos del mismo. Todas las ideas y mecánicas se han plasmado el en Documendo de Diseño del Videojuego o GDD.

- Gestión del proyecto. Desde el comienzo del proyecto se han tenido que planificar y gestionar las diferentes fases del mismo, estableciendo tareas que realizar y tiempos de desarrollo. Si bien al principio se subestimó el tiempo que se tarda en desarrollar cada parte del proyecto, lo que provocó retrasos en las fechas de entrega, se ha aprendido de los errores y conforme se ha ido avanzando en el proyecto se han ido mejorando las estimaciones y los tiempos de entrega.

- Se ha mejorado el dominio del lenguaje ensamblador. Pese a que ya se conocía el lenguaje ensamblador gracias a la asignatura de Videojuegos 1 del cuatro curso de Ingeniería Multimedia, el desarrollo de este proyecto ha permitido mejorar dicho conocimiento en cuanto a la optimización y estructuras de datos dentro de las limitaciones del lenguaje.

- Se ha aprendido el funcionamiento de la Game Boy. En cuanto al aprendizaje sobre el sistema, se ha obtenido todo el conocimiento necesario para poder realizar el proyecto y sus diferentes sistemas. No ha sido fácil, pues al comienzo del desarrollo no se tenia conocimiento alguno sobre las características de la Game Boy, pero con el tiempo se han ido mejorando hasta llegar al punto actual.

- Prototipo jugable de Project Rogue. Se ha conseguido desarrollar un prototipo jugable de Project Rogue capaz de ejecutarse en un sistema original. Este prototi-

po incluye las características principales establecidas en el apartado de objetivos, aunque no incluye todas las descritas en el Documento de Diseño del Videojuego.

Pese a que el resultado se considera altamente positivo, durante el desarrollo se han diseñado numerosas mecánicas que no han podido ser implementadas debido a la envergadura del proyecto y a los recursos disponibles. Pero de esta experiencia se ha aprendido a valorar mejor los recursos de los que se dispone y, en futuros proyectos, se aplicará el conocimiento adquirido para que el producto final sea de la mayor calidad posible.

En resumidas cuentas, el proyecto se ha realizado con éxito y ha servido para aprender mucho sobre el desarrollo y gestión de videojuegos. De ahora en adelante se aplicará todo lo aprendido durante el desarrollo de Project Rogue para la creación de futuros proyectos.



Figura 7.1.: Fotografía del juego siendo ejecutado en una Game Boy Advance SP.

Bibliografía

[1] N. Esposito, "A short and simple definition of what a videogame is," 2005.

[2] N. of Europe GmbH, "Datos técnicos — game boy / pocket / color — atención al cliente — nintendo." https://www.nintendo.es/Atencion-al-cliente/Game-Boy-Pocket-Color/Informacion-del-producto/Datos-tecnicos/Datos-tecnicos-619585.html.

[3] Keith, "(almost) z80 assembly programming for the gameboy and gameboy color." https://web.archive.org/web/20190918043456/https://www.chibiakumas.com/z80/Gameboy.php, Agosto 2018.

[4] A. J. Bentley, V. Nossum, J. Lloyd, and C. Sørensen, "Rgbds — rednex game boy development system." https://web.archive.org/web/20191217205817/https://rednex.github.io/rgbds/, Enero 2018.

[5] fsd66, "Python dungeon generator." https://github.com/fsd66/py-dungeon.git, Feb. 2020.

[6] H. Pulido, "Procedural dungeon generation algorithm with unity." https://github.com/HectorPulido/Procedural-Dungeon-Generation-Algorithm-Unity.git, Dec. 2017.

[7] Nintendo, "Game boy programming manual," Marzo 1999.

[8] P. of Anthrox, M. F. GABY, P. Felber, P. Robson, M. Korth, kOOPa, and Bowser, "Game boy cpu manual." https://web.archive.org/web/20200216205733/http://marc.rawer.de/Gameboy/Docs/GBCPUman.pdf, Marzo 1999.

[9] M. Fayzullin, "Gameboy programming info." https://web.archive.org/web/20191215214751/https://fms.komk Marzo 1999.

A. Anexo

A.1. Argumentos de RGBASM

A continuación, se van a mostrar todos los argumentos que se pueden usar a la hora de ensamblar un archivo empleando RGBASM. Toda la información ha sido obtenida de la documentación de RGBDS [4]. La estructura a seguir con los argumentos es la siguiente:

Listado A.1: Estructura del comando rgbasm

```
1 rgbasm [-EhLVvw] [-b chars] [-D name[=value]] [-g chars] [-i path] [-M dependfile
    ] [-o outfile] [-p pad_value] [-r recursion_depth] file
```

A.2. Argumentos de RGBLINK

Blablabla. Toda la información ha sido obtenida de la documentación de RGBDS [4]. La estructura a seguir con los argumentos es la siguiente:

Listado A.2: Estructura del comando rgblink

```
1 rgblink [-dtVw] [-m mapfile] [-n symfile] [-O overlayfile] [-o outfile] [-p
    pad_value] [-s symbol] [-l linkerscript] file
```

A.3. Argumentos de RGBFIX

Blablabla. Toda la información ha sido obtenida de la documentación de RGBDS [4]. La estructura a seguir con los argumentos es la siguiente:

Listado A.3: Estructura del comando rgblink

```
1 rgbfix [-CcjsVv] [-f fix_spec] [-i game_id] [-k licensee_str] [-l licensee_id] [-
    m mbc_type] [-n rom_version] [-p pad_value] [-r ram_size] [-t title_str] file
```

Tabla A.1.: Argumentos de RGBASM.

Argumento	Descripción
-b chars	Cambia los dos caracteres empleados para las constantes binarias. Por defecto son 01
-D name[=value]	Añade un símbolo de texto (string) al código fuente compilado. Es equivalente a poner [name EQUS "value"] en el código. Si no hay un valor especificado, se le da el valor 1.
-E	Exporta todas las etiquetas, incluidas las que no están referenciadas y las locales.
-g chars	Cambia los cuatro caracteres que se usan en las constantes binarias. Por defecto son 0123.
-h	Por defecto, RGBASM inserta un 'nop' después de cada 'halt'. Poner la instrucción -h lo desactiva.
-i path	Añade un 'include path'.
-L	Desactiva la optimización que cambia los 'LD [FF00+n8],A' en 'LDH [FF00+n8], A', para tener un control total del resultado.
-M dependfile	Imprime las dependencias de 'make(1)' en el archivo 'dependfile'
-o outfile	Genera un fichero binario (objeto) con el nombre especificado.
-p pad_value	Establece el relleno de una imagen. Por defecto es 0x00
-r recursion_depth	Especifica la profundidad de recursión a la que RGBASM asumirá que está en un bucle infinito.
-V	Escribe la versión del programa y sale.
-v	Se detallado.
-w	Desactiva los avisos (warnings).

Tabla A.2.: Argumentos de RGBLINK.

Argumento	**Descripción**
-m mapfile	Genera un archivo de mapeado con el nombre especificado.
-n symfile	Genera un archivo de símbolos con el nombre especificado.
-O overlayfile	
-o outfile	Genera la ROM con el nombre especificado.
-p pad_value	Establece el relleno de una imagen. Por defecto es 0x00
-w	Expande la WRAM0 de los 4KiB iniciales a los 8KiB que tiene de máximo la WRAM y prohibe el uso de secciones WRAMX.
-d	Activa el modo DMG. Prohibe el uso de secciones que no existen en un DMG tales como WRAMX y el banco 1 de la VRAM. Esta opción habilita automáticamente '-w'.
-t	Expande la ROM0 de los 16KiB iniciales a los 32KiB que tiene de máximo la ROM y prohibe el uso de secciones ROMX. Útil para ROMs que caben en 32KiB.
-l linkerscript	Establece un 'script' que le indica al 'linker' como se deben posicionar las secciones en ROM. Este archivo tiene prioridad ante el código fuente, pero debe ser consistente.
-V	Escribe la versión del programa y sale.

Tabla A.3.: Argumentos de RGBFIX.

Argumento	Descripción
-C	Establece la 'flag' GameBoyColor-Only [0x143 = 0xC0].
-c	Establece la 'flag' GameBoyColor-Compatible [0x143 = 0x80]. Si -C y -c están declaradas a la vez, -C tiene la preferencia.
-f fix_spec	Arregla ciertos valores de la cabecera que la Game Boy comprueba. También puede destruir estos valores escribiéndolos al revés. 'fix_spec' es un texto que contiene cualquier combinación de los siguientes caracteres:
-i game_id	Sustituye el texto referente al ID del juego [0x13F - 0x142] por uno de exactamente 4 caracteres. Si están definidos este y el título, el ID solapará porción del título.
-j	Establece el 'flag' regional a no japonés [0x14A = 1].
-k licesee_str	Establece el texto de distribución [0x144 - 0x145] por el proporcionado de un máximo de 2 caracteres.
-l licesee_id	Establece el antiguo ID de distribución [0x147] por el dado en el intervalo de [00x00 a 0xFF]. Este valor está obsoleto por lo que se debe poner a [0x33] en todo software moderno.
-m mbc_type	Establece el tipo de MBC [0x147] a un valor de entre [0x00 - 0xFF].
-n rom_ver	Establece la versión de la ROM [0x14C] a un valor de entre [0x00 - 0xFF].
-p pad_value	Rellena la imagen para que alcance un tamaño válido. 'pad_avlue' debe valer de [0x00 - 0xFF]. RGBFIX eligirá un tamaño de imagen válido [32 KiB, 64 Kib, 128 Kib, ...]. El byte del tamaño del cartucho [0x148] se modificará con el nuevo valor.
-r ram_size	Establece el tamaño de la RAM [0x149] a un valor de entre [0x00 - 0xFF].
-s	Establece la 'flag' SGB [0x146 = 3].
-t title	Establece el texto del título [0x134 - 0x143] de un máximo de 16 caracteres. Es recomendable usar 15 caracteres para evitar que se solape con la 'flag' CGB. Si están definidos este y el ID, el ID solapará porción del título.
-V	Escribe la versión del programa y sale.
-v	Equivale a -f lhg.

l	Arregla el logo de Nintendo
L	Destruye el logo de Nintendo
h	Arregla el checksum de cabecera [0x14D]
H	Destruye el checksum de cabecera
g	Arregla el checksum global [0x14E - 0x14F]
G	Destruye el checksum global